

DATA 298B

MSDA Project II

Final Report

Team 3

Aparna Bharathi Suresh

Pavan Srivatsav Devarakonda

Sai Naga Sanjana Chippada

Shravani Dattaram Gawade

Sujata Joshi

Department of Applied Data Science

San Jose State University

DATA 298B: MSDA Project II

Dr. Simon Shim

December 05, 2025

Abstract

Chronic diseases, such as cardiac conditions, respiratory diseases and neurological disorders, demand continuous monitoring and patient engagement, yet healthcare systems face resource constraints that limit timely assistance. The proposed solution seeks to bridge the gap between healthcare demand and service availability, ultimately improving patient outcomes and overall satisfaction. This project develops a patient-focused healthcare chatbot for chronic disease management, leveraging large language models (LLMs) and Retrieval-Augmented Generation (RAG). Open-source healthcare-specialized LLMs, including **BioMistral-7B**, **MedLlama3**, **MedAlpaca**, and **MediTron**, are fine-tuned on datasets from **MIMIC-III/IV (PhysioNet)**, **iCliniq (Hugging Face)**, **PubMedQA**, and **MedDialog**. The system is deployed on **Google Cloud Platform (GCP)** with an intuitive front-end interface using Streamlit allowing patients to seamlessly access and explore information. An **Extract-Load-Transform (ELT)** pipeline processes patient and medical data, with role-based access control (RBAC) ensuring HIPAA compliance. The application enhances patient engagement with on-demand access, reduces unnecessary hospital visits, and supports healthcare efficiency by automating routine queries. The chatbot can be integrated into hospitals, telemedicine and self-care management, ultimately improving patient outcomes and healthcare efficiency. Performance is evaluated using **LLM Judge**, **GEval**, **Factual Consistency Score**, **BERT-BLEU Hybrid** and **weighted average of MCR, METEOR, ROUGE** to ensure accurate and safe response.

Table of Contents

1. Introduction.....	5
1.1 Project Background and Executive Summary.....	5
1.2 Project Requirements	5
1.3 Project Deliverables	6
1.4 Technology and Solution Survey	7
1.5 Literature Survey of Existing Research.....	8
2. Data and Project Management Plan.....	12
2.1 Data Management Plan.....	12
2.2 Project Development Methodology	14
2.3 Project Organization Plan	18
2.4 Project Resource Requirements and Plan	18
2.5 Project Schedule	19
3. Data Engineering	20
3.1 Data Process.....	20
3.2 Data Collection	20
3.3 Data Pre-processing	20
3.4 Data Transformation	22
3.5 Data Preparation	22
3.6 Data Statistics.....	23
3.7 Data Analytics Results	24
4. Model Development.....	26
4.1 Model Proposals	26
4.2 Model Supports	29
4.3 Model Comparison and Justification	32
4.4 Model Evaluation Method.....	35
4.5 Model Validation and Evaluation Results	37
5. Data Analytics System.....	38

5.1 System Requirements Analysis.....	38
5.2 System Design	40
5.3 Intelligent Solution.....	42
5.4 System Supporting Environment.....	46
6. System Evaluation and Visualization.....	50
6.1 Analysis of Model Execution and Evaluation Results.....	50
6.2 Achievements and Constraints	54
6.3 System Quality Evaluation of Model Functions and Performance:	62
6.4 System Visualization.....	65
7. Conclusion	73
7.1 Summary.....	73
7.2 Benefits and Shortcomings	73
7.3 Potential System and Model Applications	74
7.4 Experience and Lessons Learned	75
7.5 Recommendations for Future Work.....	75
7.6 Contributions and Impacts on Society.....	76
References	77
Appendices.....	80

1. Introduction

1.1 Project Background and Executive Summary

Chronic diseases, such as Cardiac Conditions, Respiratory Diseases, and Neurological Disorders, require continuous monitoring and patient engagement. Our project aims to solve the concerns related to the lack of timely medical assistance to patients and reducing the workload for healthcare professionals. Through this project, we are developing a Generative AI Healthcare chatbot, offering a solution that combines a Retrieval-Augmented Generation (RAG) framework with large language models.

Symptom Checker RAG - This RAG helps users assess their symptoms, explore possible conditions, ask follow-up questions, and determine whether professional care may be needed. It is useful for managing ongoing health concerns, including conditions with overlapping or complex symptoms, by providing timely and appropriate guidance.

The rag has utilized open-source, healthcare-specialized LLM **MedLlama3**, fine-tuned on high-quality datasets such as **MIMIC- III**, **PubMedQA**, and **MedDialog**. Real-time integration with medical data sources, medical repositories, and textbooks ensuring the system remains current and relevant. The solution is deployed on **Google Cloud Platform** for scalability and reliability, with a user-friendly chatbot interface developed using **Streamlit** to enhance accessibility.

1.2 Project Requirements

The **Generative AI Healthcare Chatbot** project requires specific functionality, performance, and technical specifications to ensure accurate and scalable healthcare support. Each requirement includes measurable criteria for evaluation.

Functional Requirements

- **Accurate Responses** - The chatbot must provide contextually relevant medical advice, measured by BLEU and ROUGE scores.
- **Real-Time Information Retrieval** - Using a Retrieval-Augmented Generation (RAG) framework, the system will retrieve medical information in real time to ensure the knowledge of relevance assessed via similarity metrics.
- **Medical Concept Recall** - The system must achieve an acceptable Medical Concept Recall (MCR) score greater than the base model.

Technical Requirements

- **Fine-Tuning LLM** - Models like BioMistral-7B, MedLlama3, MedAlpaca and MediTron are fine-tuned, with performance evaluated by LLM Judge, GEval, Factual Consistency Score, BERT-BLEU Hybrid and weighted average of MCR, METEOR, ROUGE on test dataset.
- **Medical Journal Integration** - Real-time retrieval is from the Medical Journals which will have knowledge relevance via similarity between user queries and retrieved data.
- **Datasets** - Fine-tuning will use MIMIC-III, MIMIC-IV, PubMedQA, and MedDialog datasets.
- **Vector Database** - Preprocessed medical knowledge will be stored in a vector database for efficient retrieval.

1.3 Project Deliverables

The Generative AI Healthcare Chatbot project will deliver the following outputs to ensure a fully functional, documented, and accessible solution for stakeholders:

- **Project Documentation** - Comprehensive documentation detailing the end-to-end development process, including model training methodologies, evaluation metrics and deployment configurations to serve as a reference for future.
- **Working Chatbot Prototype** - A fully operational chatbot prototype with a robust backend, capable of retrieving medical information.
- **Web-Based Chatbot Interface** - A web-based interface built using Streamlit, providing an intuitive and accessible platform for patients to interact with the chatbot.
- **Cloud-Deployed Chatbot Service** - A production-ready chatbot service deployed on Google Cloud Platform (GCP).

1.4 Technology and Solution Survey

A survey of current technologies including a summary of classifications of features and applications.

Technology	Feature	Applications
Large Language Models	To work on vast amounts of textual data for text generation, understanding, translation, and NLP tasks.	Chatbots, virtual assistants, language translation services, question-answering systems.
Medical-Specific Models	Models like MedAlpaca and Meditron to handle specialized medical terminologies & healthcare knowledge.	Clinical decision support, electronic health record analysis, medical information extraction.

Real-Time Knowledge Retrieval	Google Bard to access real-time information to enhance the accuracy of responses.	Search engines, news aggregation, dynamic knowledge retrieval systems.
Vector Database Integration	Tools like Chroma db, Pinecone, Weaviate and FAISS to store & manage vectorized data for similarity retrieval.	Recommendation systems, semantic search, anomaly detection.
HIPAA-Compliant Services	GCP Comprehend Medical and Google Cloud Healthcare API for sensitive medical data.	Medical text analysis, patient data extraction, healthcare insights, secure data processing.
Open-Source Customization	Hugging Face and OpenNLP to customize solutions for model training & fine-tuning.	Custom NLP applications like text classifiers, sentiment analyzers, entity recognition systems.

1.5 Literature Survey of Existing Research

“The Effectiveness of Artificial Intelligence Conversational Agents in Health Care: Systematic Review” authored by Madison Milne-Ives¹ et.al^[1] assessed the effectiveness and usability of conversational agents in health care, finding mostly positive or mixed results.

Justification: Further research is needed to improve study designs, address privacy and security concerns, and evaluate cost-effectiveness to ensure sustainable adoption in health care.

“A survey on agents’ applications in healthcare: Opportunities, challenges and trends” written by Emilio Sulis et.al[2] survey examines the application of agent-based technologies in healthcare, highlighting opportunities, challenges, and trends. **Justification:** The findings underscore the potential of agent-based approaches to enhance healthcare efficiency and decision-making, but also reveal challenges like model validation, system interoperability, and trust.

“A Systematic Review on Healthcare Artificial Intelligent Conversational Agents for Chronic Conditions” authored by Bhuvana Narayan et al.[3] examines the role of AI-powered conversational agents in managing chronic conditions, analyzing their effectiveness, usability, and AI methods used. **Justification:** The study highlights the potential of conversational agents in chronic disease management but calls for improved study designs, standardized evaluation metrics, and further research on their long-term effectiveness, security, and cost-efficiency.

“Redefining Medicine: The Power of Generative AI in Modern Healthcare” authored by Chettim Chetty Hemasri et al.[4] explores how generative AI (GAI), including LLMs like GPT-4 and Med-PaLM, is transforming healthcare by improving diagnostics, drug discovery, and patient engagement. **Justification:** While GAI has immense potential to revolutionize healthcare, addressing privacy, bias, and explainability issues is crucial for its safe and effective integration into medical practice.

The paper **"Generative AI for Transformative Healthcare"** by Siva Sai et al.[5] provides a comprehensive exploration of how generative artificial intelligence (GAI) models, such as ChatGPT and DALL-E, are revolutionizing healthcare through applications like medical imaging,

drug discovery, personalized treatment, and clinical trial optimization. **Justification:** This study is significant as it fills a research gap by offering a detailed analysis of GAI’s potential and challenges in healthcare, supported by real-world examples and evaluations.

The paper “**Natural Language Programming in Medicine: Administering Evidence Based Clinical Workflows with Autonomous Agents Powered by Generative Large Language Models**” by Akhil Vaid MD et al.[8]. The paper investigates the use of generative Large Language Models (LLMs) as autonomous agents, utilizing real-world clinical cases, and employing techniques like RAG to enhance decision-making. **Justification:** This research is significant as it demonstrates the potential of LLMs to emulate physician behavior in complex healthcare settings while identifying areas for refinement in model performance and integration.

Comparison of relevant papers

Paper	Focus	Techniques	Applications	Challenges	Key Findings
The Effectiveness of Artificial Intelligence Conversational Agents in Health Care: Systematic Review	Conversational Agents in healthcare	NLP, chatbots, embodied conversational agents	Mental health, clinical decision support, patient education	Limited understanding, interactivity, cost-effectiveness	Positive or mixed effectiveness and usability
A survey on agents applications in healthcare: Opportunities, challenges and trends	Agent-based techniques in healthcare	Multi-agent systems, agent-based modeling	Distributed decision-making, epidemiology, patient	Trust, explainability, reliability, integration with ML and IoMT	Identifies research clusters and trends in agent-based healthcare

			empowerment		
A Systematic Review on Healthcare Artificial Intelligent Conversational Agents for Chronic Conditions	CAs for chronic conditions	NLP, speech recognition, machine learning	Chronic disease management, patient education	Limited technical reporting, heterogeneity in studies	Promising user acceptance, need for more robust evaluations
Redefining Medicine: The Power of Generative AI in Modern Healthcare	Generative AI in healthcare	GANs, LLMs, GPT-based systems	Drug development, medical imaging, personalized treatment	Data security, ethical considerations, regulatory compliance	GAI enhances diagnostic accuracy and accelerates drug discovery

2. Data and Project Management Plan

2.1 Data Management Plan

Data Collection Approaches

Primary Datasets

- **MIMIC-IV**: A comprehensive dataset of de-identified clinical records from intensive care units, including patient information, treatments, diagnoses, and outcomes.
- **MIMIC-III**: Historical clinical data complementing MIMIC-IV, providing additional context for model training.
- **PubMedQA**: A dataset of biomedical question-answering pairs to enhance the chatbot's ability to respond to medical queries.
- **iCliniq**: A dataset of patient-doctor question-and-answer interactions, providing real-world medical queries and responses to support chatbot training.

Real-Time Updates:

- Retrieved documents are stored in a ChromaDB database for efficient access by the Retrieval-Augmented Generation (RAG) framework.

Collection Methods:

- Data extracted from PhysioNet for MIMIC-III, MIMIC-IV.
- Direct database queries extract historical data from PubMedQA and iCliniq.
- File-based extraction processes bulk data in CSV and JSON formats.

Data Management Methods:

Preprocessing and Normalization:

- Automated Python pipelines, leveraging Pandas, NumPy and Scikit-learn, handle data cleaning and transformation.
- Missing value imputation replaces or removes incomplete entries to ensure consistency. Numerical fields are normalized, and categorical variables are uniformly encoded to minimize bias and enhance data integrity for model training.

Secure Storage Solutions

- **Cloud Storage:** Data is stored on Google Cloud Platform (GCP) for high availability and scalability. Automated daily backups and disaster recovery protocols are maintained using GCP's storage solutions.
- **Encryption:** Data is encrypted in transit and at rest to protect against unauthorized access.
- **Access Control:** Role-based access control (RBAC) and audit logs restrict and monitor access to sensitive data.
- **Compliance:** Data handling complies with regulations, such as HIPAA, to ensure ethical and legal standards are met.

Usage Mechanisms

- **Model Training:** Preprocessed datasets (MIMIC-III, MIMIC-IV, PubMedQA, iCliniq) are used to fine-tune large language models for accurate medical responses.
- **Medical Journals:** a synchronized training mechanism, enhances the RAG framework by jointly fine-tuning the LLM and information retrieval system.
- **Chatbot Functionality:** Processed data and real-time retrieval of data enabling accurate and timely responses to user queries with clear user-interface.

2.2 Project Development Methodology

The **Generative AI Healthcare Chatbot** project follows a structured methodology to develop an intelligent, data-driven chatbot for chronic disease management. This section outlines the **Data Analytics with Intelligent System Development Life Cycle (SDLC)** and the **Planned Project Development Processes and Activities**, ensuring a robust and scalable solution.

Data Analytics with Intelligent System Development Life Cycle

The project adopts a tailored SDLC to integrate data analytics and intelligent system development, with the following stages:

- **Requirement Analysis & Specification** - Gathered functional and non-functional requirements, focusing on clinical needs and operational constraints. Define key performance indicators (KPIs) such as Factual Consistency Score, Medical Concept Recall (MCR), and GEval to evaluate chatbot performance.
- **Data Collection & Preprocessing** - Collected data from PubMedQA, MIMIC-III, MIMIC-IV, and iCliniq, with real-time updates from PubMed. Datasets underwent preprocessing workflows, including cleaning, normalization and transformation, using Python tools (Pandas, NumPy, Scikit-learn). Datasets are processed individually for specific tasks and merged where appropriate to enhance model generalization.
- **Model Development & Fine-Tuning** - Fine-tuned open-source large language models (LLMs), including BioMistral-7B, MedLlama3, MedAlpaca and MediTron, using the preprocessed datasets. Applied Retrieval-Augmented Generation (RAG) framework, storing vectorized data in a vector database for efficient retrieval.
- **Integration & System Development** - Developed an end-to-end healthcare chatbot by integrating fine-tuned models with a chatbot interface built using Streamlit. Deploy the

system on Google Cloud Platform (GCP), ensuring scalability, security and compliance with medical data regulations.

- **Testing, Evaluation & Iteration** - Conducted comprehensive testing (unit, integration, usability). Evaluated performance against KPIs and iterated based on responses and system metrics to optimize accuracy and responsiveness.
- **Deployment & Monitoring** - Deployed the solution on GCP with continuous monitoring for scalability, model drift, security vulnerabilities and compliance.

Planned Project Development Processes and Activities

The project is structured into phased processes with defined activities and deliverables:

1. Initiation and Planning

1. Conducted kickoff meetings, gathered requirements, and assessed risks.
2. Finalized data sources
3. **Deliverables:** Project charter, initial project plan, risk assessment document.

2. Data Management and Preprocessing

1. Extracted, cleaned and normalized data from PubMedQA, MIMIC-III, MIMIC-IV and iCliniq
2. Developed and validated preprocessing pipelines.
3. **Deliverables:** Processed datasets, data dictionaries, preprocessed documentation.

3. Model Development and Fine-Tuning

1. Fine-tuned LLMs using datasets.
2. Integrated RAG with VectorDB.

3. **Deliverables:** Fine-tuned models, training scripts, performance evaluation reports.

4. System Integration and Interface Development

1. Integrated models with the Streamlit-based chatbot interface.
2. **Deliverables:** Integrated application prototype, user interface design documents, integration test reports.

5. Testing, Evaluation, and Deployment

1. Performed unit, integration, and usability testing.
2. **Deliverables:** Testing reports, deployment scripts, live system documentation.

6. Maintenance and Continuous Improvement

1. Monitored system performance and compliance.
2. **Deliverables:** Maintenance logs, periodic review reports.

Data Pipeline Architecture

Data Sources

- **MIMIC-III & IV:** Sourced from PhysioNet as SQL databases, containing clinical notes, treatments, and outcomes.
- **PubMedQA:** Biomedical question-answering pairs, accessed as CSV files from public repositories, to enhance query response capabilities.
- **iCliniq:** Patient-doctor Q&A interactions, obtained from Hugging Face as CSV files, providing real-world medical queries.

ELT Processes

- **Extraction:** Ingest data from SQL databases (MIMIC-III, MIMIC-IV), APIs (PubMed), and local files (PubMedQA, iCliniq). Automated Python scripts manage version control and scheduling to preserve data lineage.
- **Loading:** Load raw data into the PostgreSQL warehouse on GCP, maintaining original formats for flexibility in subsequent transformations.
- **Transformation:**
 - 1.1 **Data Cleaning & Normalization:** Use Pandas, NumPy and Scikit-learn within the warehouse to address missing values, duplicates, and inconsistencies in patient and medical data.
 - 2.1 **Standardization:** Apply uniform encoding for categorical variables and normalization for numerical features.
 - 3.1 **Enrichment:** Merge metadata (e.g., ICD codes) from external knowledge bases when needed.

Data Transfers with Pipeline

- **Automated Pipelines:** Orchestrated using Apache Airflow to manage dependencies, scheduling, and monitoring. Patient and medical data is processed for the Retrieval-Augmented Generation (RAG) framework, with vectorized data.
- **Incremental Updates:** Process only new or updated records to optimize performance for real-time patient query responses.
- **Monitoring & Logging:** Track data movement, errors and performance with comprehensive logs.

2.3 Project Organization Plan

Work Breakdown Structure (WBS)

The project is decomposed into six primary phases, each containing specific deliverables and work packages (WPs) to ensure a structured, incremental, and accountable approach:

Phase 1.0: Initiation, Deliverables: Project charter, initial project plan, communication plan.

Phase 2.0: Data Management, Deliverables: Processed datasets, data dictionaries, preprocessed documentation.

Phase 3.0: Model Development, Deliverables: Fine-tuned models, training scripts, performance evaluation reports.

Phase 4.0: System Integration & Interface Development, Deliverables: Integrated application prototype, user interface design documents, integration test reports.

Phase 5.0: Testing & Deployment, Deliverables: Testing reports, deployment scripts, live system documentation.

Phase 6.0: Maintenance & Support, Deliverables: Maintenance logs, performance dashboards, periodic review reports.

2.4 Project Resource Requirements and Plan

The project requires a robust set of hardware, software and tools to develop and deploy a patient- focused, AI-driven chatbot for chronic disease management.

Hardware Requirements

GCP Cloud Instances with GPU Support: High-performance Google Cloud Platform (GCP) instances are recommended for computationally intensive tasks like deep learning model

training and inference. Instances should include multi-core CPUs, a minimum of 128GB RAM, and NVIDIA A100 GPUs for optimal performance. These configurations provide scalable, on-demand computing power, accelerating training of models (e.g., BioMistral-7B, MedLlama3) on datasets like MIMIC-III/IV (PhysioNet) and iCliniq (Hugging Face), reducing latency for real-time patient query responses, and supporting iterative experimentation.

Software & Tools

- **Programming and Machine Learning Frameworks** – Python, PyTorch
- **Database Management:**
 - **GCP:** Relational database for the ELT pipeline, storing raw and transformed patient data as the single source of truth.
 - **Vector Database:** Vector database for managing high-dimensional embeddings of Medical Journal data within the RAG framework.
- **Front-End Development** – Streamlit

2.5 Project Schedule

298A – We did the data collection, data preprocessing and created a pipeline to store the transformed data in GCP.

298B – We have used that data to fine tune 4 LLMs and evaluated it and created a full stack application with the best performing model and built RAG on top of it for more accurate answers.

3. Data Engineering

3.1 Data Process

The data process starts with collecting data from various sources such as MIMIC-IV, MedQuAD and iCliniq. The data was pre-processed and cleaned by handling the missing values, duplicates, and ensuring the uniformity of data types. As part of the data cleaning phase, first, null values were imputed or removed, depending on their context and impact on data quality. Next, standardization techniques were applied, such as converting the name column to lowercase and trimming any leading or trailing whitespace, to maintain consistency across textual entries. We transformed all 4 datasets into a single Q&A dataset. For training, the dataset is divided into three subsets where training is 70%, validation is 15%, and testing is 15%.

3.2 Data Collection

- MIMIC-III/IV were downloaded from PhysioNet after completing an assessment, we were given access to it.
- iCliniq and PubMedQA are from Hugging Face.

3.3 Data Pre-processing

iCliniq

- Dropped rows with missing values in the Question or Answer columns.
- To enrich the context of each question, we concatenated the Title and Question fields into a single unified field called `full_question`.
- We created a custom text-cleaning function that converted all characters to lowercase, removed newline characters, normalized whitespace, and stripped special characters,

keeping only letters, numbers, and basic punctuation such as commas, periods, and question marks.

MedquAD

- Removed duplicate entries and any rows with missing questions or answer fields.
- Applied a similar text-cleaning function as used in the iCliniq dataset, ensuring all text was lowercase, whitespace was standardized, and irrelevant symbols were removed.

MIMIC III/IV

- There were multiple tables in this dataset, we merged all the tables together and removed unwanted columns and made it into a single table.
- Standardized the name field (drug name) by removing trailing spaces and converting the text to lowercase.
- Timestamp fields like chart time were converted into standard datetime formats for future chronological analysis.
- Merged ICD data set to this finally to understand the codes.

After processing all the datasets individually, we merged all the datasets into one single Q and A dataset.

Data Pipeline Demo

We orchestrated data pipeline via Apache Airflow, managed by Cloud Composer on GCP, with three modular DAGs handling distinct tasks: merging, preprocessing, and exploratory data analysis (EDA) with ICD code enrichment.

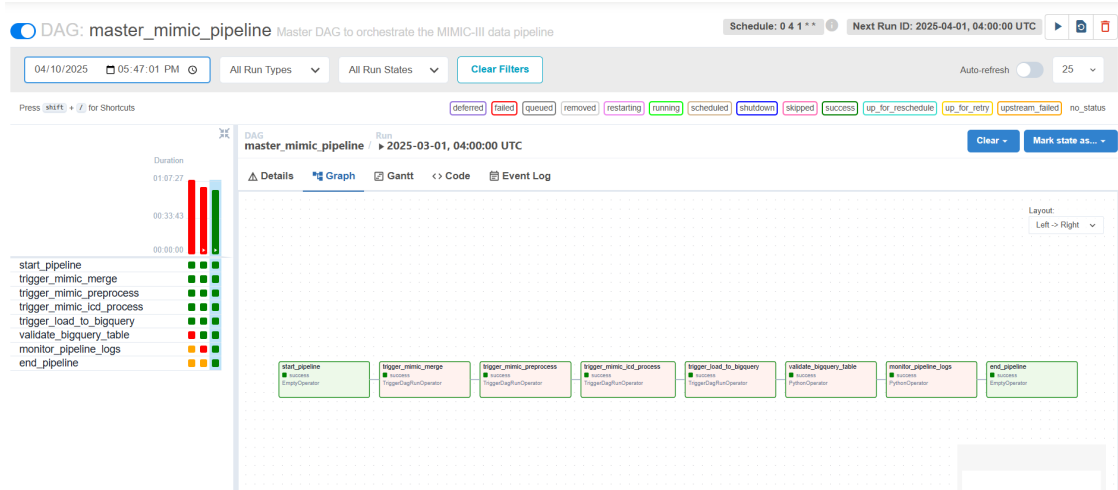


Figure – Data Pipeline

3.4 Data Transformation

Merged file as the output of DAGs was created in the GCP bucket

The screenshot shows the Google Cloud Storage interface. The left sidebar contains navigation links for Cloud Storage, Overview, Buckets, Monitoring, Settings, Storage Intelligence, Insights datasets, and Configuration. The main area displays the 'Bucket details' for 'data298a'. Under the 'Objects' tab, a folder browser shows the hierarchy: data298a > final > merged_final.csv. A table lists the objects, showing 'merged_final.csv' with a size of 2.1 GB, type of text/csv, and created on Apr 9, 2025, 2:40:29 AM.

Name	Size	Type	Created	Storage class	Last r
merged_final.csv	2.1 GB	text/csv	Apr 9, 2025, 2:40:29 AM	Standard	Apr 9,

Figure – Transformed data in Bucket

3.5 Data Preparation

To align with LLM training requirements, the extracted data was transformed into the JSONL (JSON Lines) format. After formatting the dataset into JSONL, we performed a stratified split into training, validation, and testing subsets using an 80-10-10 ratio. This structured approach

to data preparation ensures consistency, scalability, and readiness for downstream model training tasks.

3.6 Data Statistics

 Raw Dataset Info: <class 'pandas.core.frame.DataFrame'> RangeIndex: 16412 entries, 0 to 16411 Data columns (total 6 columns):				 Preprocessed MedQuAD Dataset Info: <class 'pandas.core.frame.DataFrame'> RangeIndex: 16359 entries, 0 to 16358 Data columns (total 7 columns):			
#	Column	Non-Null Count	Dtype	#	Column	Non-Null Count	Dtype
0	question	16412 non-null	object	0	question_clean	16359 non-null	object
1	answer	16407 non-null	object	1	answer_clean	16359 non-null	object
2	source	16412 non-null	object	2	focus_area	16345 non-null	object
3	focus_area	16398 non-null	object	3	focus_area_encoded	16359 non-null	int64
4	Question_Length	16412 non-null	int64	4	tokenized	16359 non-null	object
5	Answer_Length	16412 non-null	int64	5	Question_Length	16359 non-null	int64
dtypes: int64(2), object(4)				6	Answer_Length	16359 non-null	int64
memory usage: 25.7 MB				dtypes: int64(3), object(4)			
				memory usage: 108.9 MB			
 Raw MedQuAD Descriptive Statistics:				 Preprocessed Descriptive Statistics:			
	Question_Length	Answer_Length			Question_Length	Answer_Length	
count	16412.000000	16412.000000		count	16359.000000	16359.000000	
mean	50.684438	1303.056483		mean	49.892475	1247.815148	
std	16.925355	1656.597408		std	17.020959	1503.019636	
min	16.000000	3.000000		min	13.000000	6.000000	
25%	38.000000	487.000000		25%	37.000000	470.000000	
50%	48.000000	889.500000		50%	48.000000	867.000000	
75%	61.000000	1589.000000		75%	60.000000	1551.000000	
max	191.000000	29046.000000		max	184.000000	26717.000000	

Figure - MedQuAD Statistics

```

Raw iCliniq Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 39540 entries, 0 to 39539
Data columns (total 5 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Title                  39540 non-null  object
1   Question               39540 non-null  object
2   Answer                 39540 non-null  object
3   Question_Length        39540 non-null  int64
4   Answer_Length          39540 non-null  int64
dtypes: int64(2), object(3)
memory usage: 61.7 MB

Preprocessed iCliniq Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 39540 entries, 0 to 39539
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   question_clean         39540 non-null  object
1   answer_clean           39540 non-null  object
2   input_ids              39540 non-null  object
3   attention_mask         39540 non-null  object
4   token_type_ids         39540 non-null  object
5   Question_Length        39540 non-null  int64
6   Answer_Length          39540 non-null  int64
dtypes: int64(2), object(5)
memory usage: 276.5 MB

Raw Text Length Statistics:
      Question_Length  Answer_Length
count  39540.000000    39540.000000
mean    525.851340      857.836495
std     444.292151      536.409209
min      13.000000       26.000000
25%     250.000000      529.000000
50%     406.000000      739.000000
75%     649.000000     1041.000000
max    11039.000000    12585.000000

Preprocessed Text Length Statistics:
      Question_Length  Answer_Length
count  39540.000000    39540.000000
mean    590.955665      856.454628
std     445.151414      535.483563
min      77.000000       26.000000
25%     315.000000      528.000000
50%     473.000000      738.000000
75%     716.000000     1039.000000
max    11143.000000    12569.000000

```

Figure - iCliniq Statistics

3.7 Data Analytics Results

MIMIC IV: The raw MIMIC-IV dataset was composed of five separate CSV files: diagnoses.csv, medications.csv, vitals.csv, admissions.csv, and patients.csv. These files collectively contained hundreds of thousands of clinical records. During preprocessing, a relational join was performed across subject_id and hadm_id keys to generate a single consolidated dataset, which was saved as preprocessed_mimic_iv.csv.

The final dataset retained 7 million rows where all critical components (ICD codes, vitals, chief complaint, and medications) were successfully aligned. This merged dataset included structured fields like icd_category, icd_chapter_combined, and chiefcomplaint, which were used for statistical analysis and tokenization.

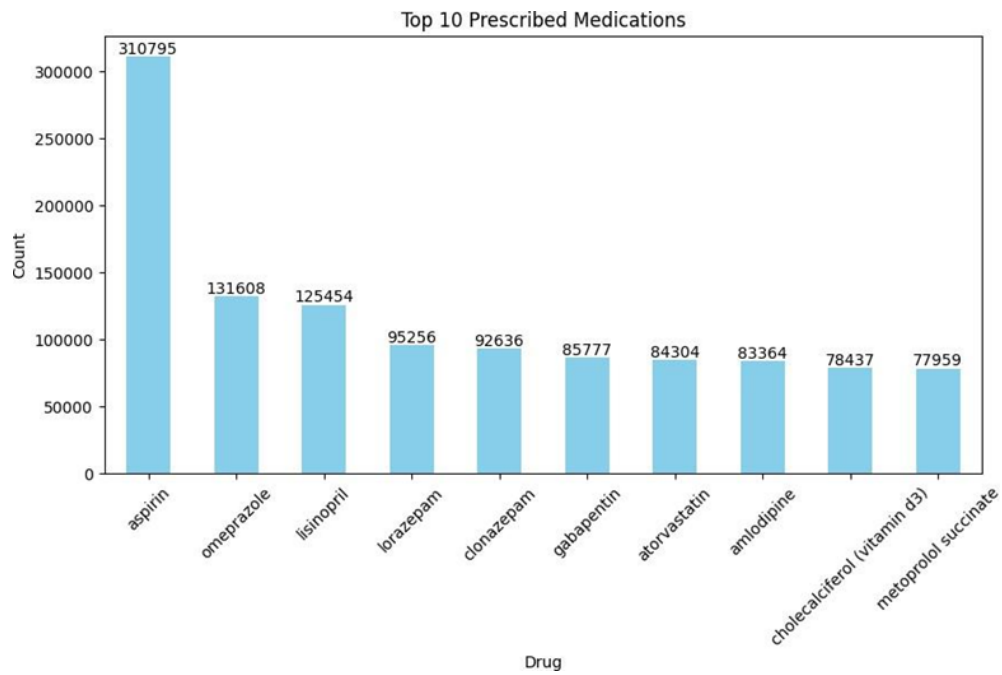


Figure – Top 10 Prescribed Medications

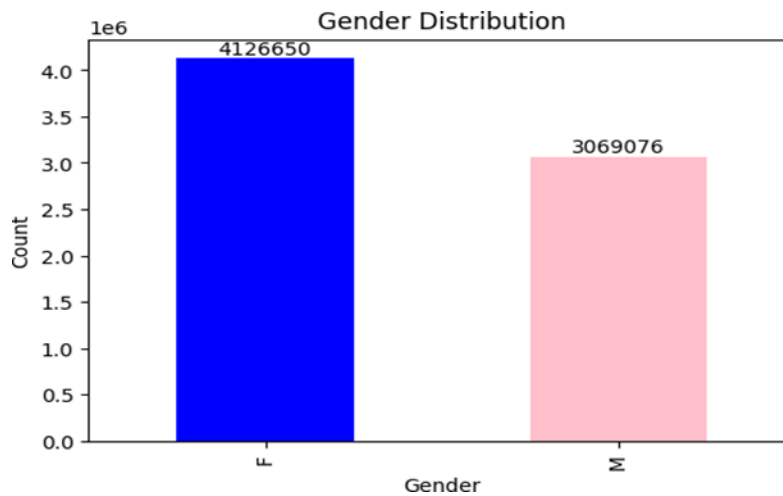


Figure – Gender Distribution

4. Model Development

4.1 Model Proposals

a) MedLlama3

MedLlama3 is a proposed adaptation of the Llama-3 language model, specifically engineered to address the unique demands of healthcare applications. As one of the proposed models, MedLlama3 is positioned to contribute to accessible and efficient health solutions. MedLlama3 is a transformer-based architecture of Llama-3, as illustrated in the figure below (sourced from Google). It is optimized for handling complex text-processing tasks, making it well-suited for medical applications that require precise and contextually relevant outputs.

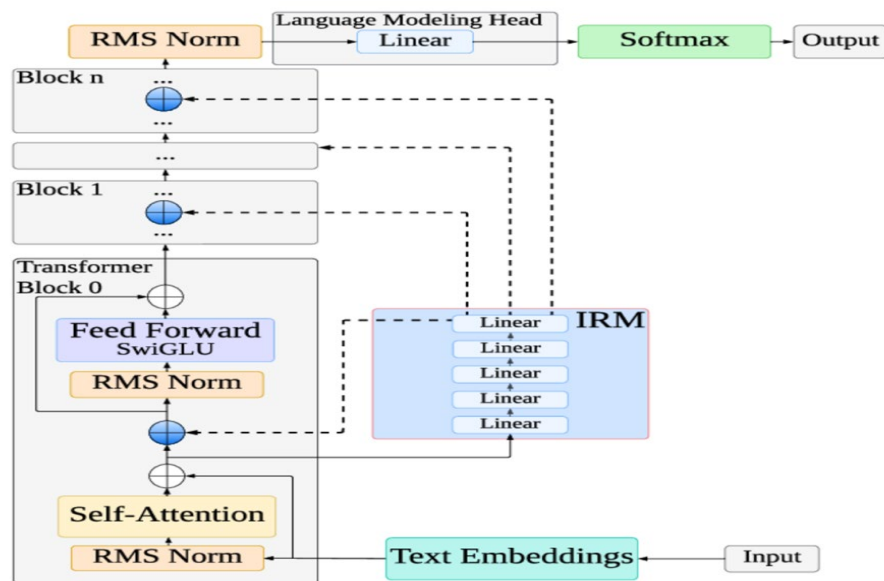


Figure - Llama3 Architecture

The architecture of MedLlama3 is composed of Text embeddings, Transformer blocks which have Self attention, Feed forward layer, and Normalization layer, Softmax that work together to process and generate relevant content. MedLlama3 is capable of performing complex

Query Processing, understanding the context, text generation, and scalable. We have chosen MedLlama3 as one of our models because it is trained on healthcare dataset and it has a ability to process complex queries.

b) BioMistral 7B

BioMistral 7B is an open-source large language model (LLM) specifically designed for biomedical and healthcare applications. The foundation model is Mistral 7B, which is a transformer-based architecture. BioMistral 7B has been pre-trained on biomedical data, which makes the model understand medical terminology, concepts, and relationships. We chose this model as it addresses the challenges of adapting general-purpose LLMs to the medical domain and performs well in medical question-answering tasks.

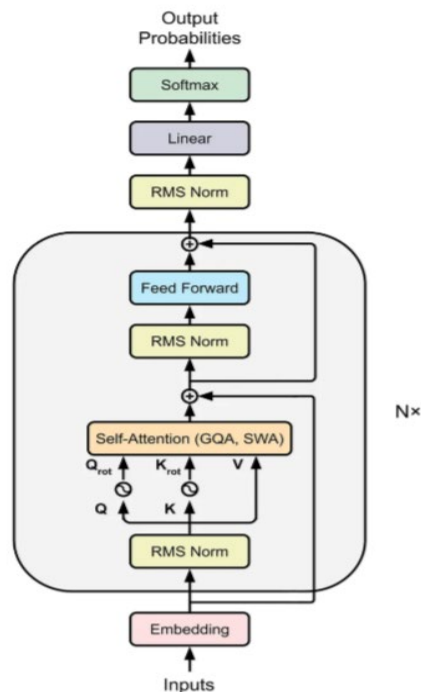


Figure - BioMistral Architecture

The architecture of BioMistral 7B has several advancements compared to the transformer model to achieve high performance, the advancements are:

- Layer normalization is replaced by RMS Normalization.
- Absolute Positional encoding is replaced by Rotary Position Embedding (RoPE).
- Multi head attention replaced by Grouped Query Attention (GQA).
- Sliding Window Attention to improve training and inference speed.
- Replaced ReLU with SwiGLU Activation Function.

c) Meditron

Meditron is also an open-source large language model specifically tailored to the medical domain. It is a transformer-based model, but it is decoder only transformer built upon the Llama-2 and Llama-3.1 architectures, Meditron includes models with 7B and 70B parameters (Meditron-7B and Meditron-70B). Pre-trained on a curated medical corpus, Meditron excels in medical reasoning tasks, outperforming other open-source models and even some closed-source commercial LLMs like GPT-3.5 and Med-PaLM on standard benchmarks. This architecture, combined with domain-specific pre-training, positions Meditron as a leading open-source medical LLM suite.

Meditron is proposed as one of the models for our project due to its exceptional performance, open-source accessibility, and alignment with the goal of advancing equitable healthcare solutions. Its ability to outperform closed-source models like GPT-3.5 and Med-PaLM on medical benchmarks demonstrates its robustness for medical tasks.

d) MedAlpaca

MedAlpaca is a transformer-based large language model optimized for biomedical and clinical text generation. Built upon Meta's LLaMA architecture, MedAlpaca follows a decoder-only transformer design similar to GPT models, but it is fine-tuned on domain-specific medical corpora to enhance performance on healthcare and life-sciences tasks. Its architecture emphasizes contextual fluency, domain precision, and scalability, making it effective for both clinical and research applications. Trained on PubMed, clinical guidelines, and Q&A datasets, to adapt general-purpose knowledge to healthcare contexts.

4.2 Model Supports

The healthcare AI assistant for chronic disease management is powered by a sophisticated integration of Google Cloud Platform (GCP) services and domain-specific open-source large language models (LLMs). This system leverages fine-tuned LLMs **MedLlama3**, **BioMistral-7B**, **MedAlpaca**, and **Meditron** to deliver accurate, contextually relevant, and medically reliable responses. Deployed on GCP with a user-friendly front-end interface built using Streamlit, the system is designed for scalability, security, and continuous improvement, with performance monitoring.

Key Technologies and Tools

1. **Google Cloud Storage (GCS)** - Serves as the primary storage for raw and processed healthcare datasets, including MIMIC-III/IV, iCliniq, PubMedQA, and MedDialog. Provides scalable, secure, and centralized data lakes, facilitating efficient data management and retrieval for model training and inference.

2. **Google Dataflow** - Executes Extract-Load-Transform (ELT) pipelines to preprocess raw healthcare data, performing cleansing, normalization, feature extraction, and aggregation. Transforms datasets like MIMIC-III/IV and iCliniq into formats suitable for model training and embedding generation, writing processed data to BigQuery. Automates scalable data processing, ensuring high-quality inputs for machine learning workflows.
3. **BigQuery** - Acts as a data warehouse for storing pre-processed, normalized datasets used for analytics, feature engineering, and model training. Enables large-scale querying and feature selection, supporting ad hoc analyses and scheduled training pipelines for LLMs. Provides fast, SQL-based access to structured data, enhancing the efficiency of model development and evaluation.
4. **Front-End Interface** - A user-friendly interface built with Streamlit allows patients and healthcare providers to interact seamlessly with the AI assistant. Provides an intuitive platform for chronic disease management, accessible via telemedicine platforms, hospital systems, or self-care applications.

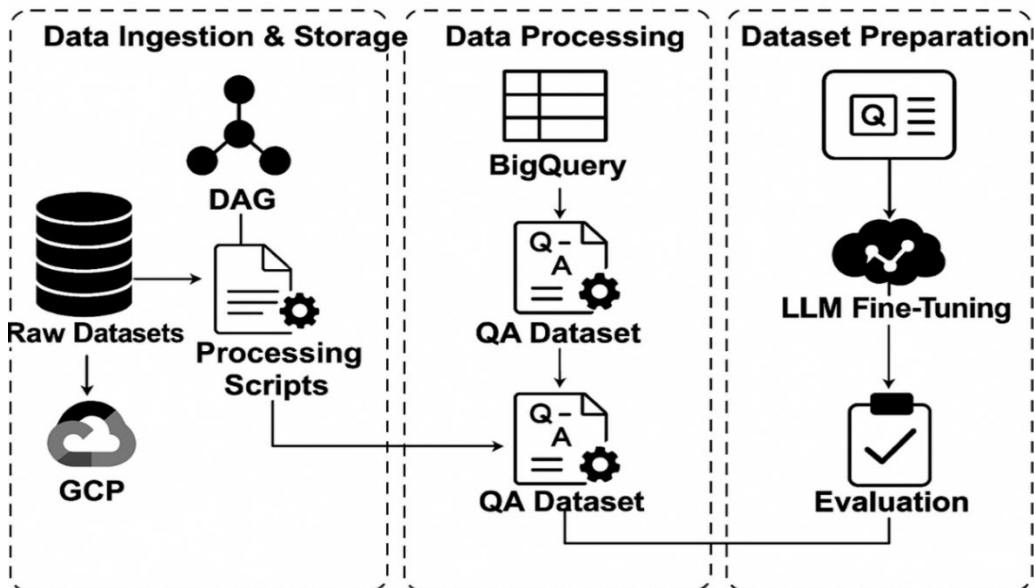


Figure: End-to-End Architecture

The figure illustrates the end-to-end data pipeline of the project. Raw medical datasets stored in GCP are processed using DAG-based scripts, and the cleaned outputs are stored in BigQuery. From there, a question-answer (QA) dataset is constructed and used to fine-tune large language models (LLMs), followed by evaluation to validate model performance.

Data Ingestion & Storage

- **Source Integration:** Supports ingestion from local folders, mounted Google Drive locations, and APIs, enabling import of clinical datasets, logs, and structured forms.
- **Scheduled Batch Jobs:** Cloud Scheduler triggers batch jobs to extract and upload data to GCS at regular intervals.
- **GCS:** Centralizes raw and semi-structured datasets in scalable, secure data lakes.
- **Dataflow for Preprocessing:** Performs ELT operations such as cleansing, normalization, and feature extraction preparing data for analytics and ML workflows.
- **BigQuery for Analytics:** Loads processed data into BigQuery for large-scale querying, feature selection, and training pipelines.

Model Training

- **Feature Engineering:** Preprocesses data for enhanced prediction of precision and contextual understanding.
- **Embedding Generation:** Uses PubMed-BERT to generate embeddings for semantic understanding, stored in Pinecone.

- **Domain-Specific Fine-Tuning:** Fine-tunes MedLlama3, BioMistral-7B, MedAlpaca, and Meditron on 4 large healthcare datasets using LoRA, specializing models for medical queries.

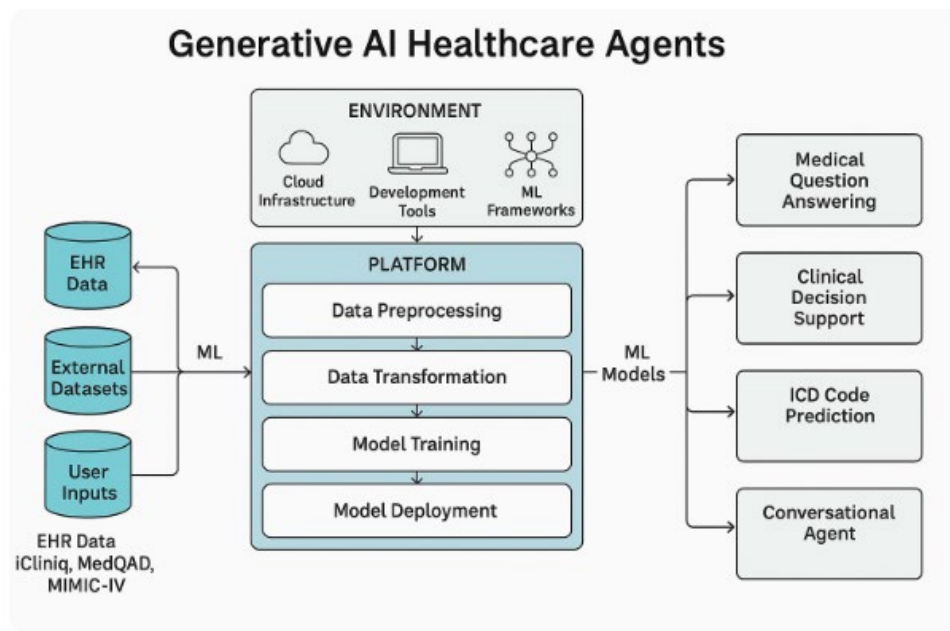


Figure - Platform architecture, components, machine learning data flows.

4.3 Model Comparison and Justification

The healthcare AI assistant for chronic disease management integrates four open-source, healthcare-specialized large language models (LLMs): MedLlama3, BioMistral-7B, MedAlpaca, and Meditron. These models were selected for their complementary strengths in addressing diverse requirements, including general medical question-answering, biomedical knowledge retrieval, generative text production, and high-performance medical reasoning. Each model is fine-tuned on datasets such as MIMIC-III/IV, iCliniq, PubMedQA, and MedDialog. The following comparison table and justification outline the targeted problems, features, approaches, strengths, limitations, and data characteristics of each model, demonstrating their collective role in achieving the

project's goals of enhancing patient engagement, reducing healthcare burden, and improving outcomes.

Model	Targeted Problem	Features	Approach	Strengths	Limitations	Data Size & Complexity	Types of Data
MedLlama3	General-purpose healthcare Q&A, patient education, preliminary health advice	Built on Llama-2; versatile in medical Q&A; handles complex medical terminology via self-attention mechanisms	Transformer-based deep learning with fine-tuning on healthcare datasets	High accuracy and versatility for diverse medical queries; ideal for general healthcare applications and patient-facing interactions	Computationally intensive; may require significant resources for real-time deployment in low-resource settings	High data size, complex queries	General medical queries, patient education materials, clinical notes
BioMistral-7B	Biomedical knowledge retrieval, clinical research Q&A, specialized medical queries	Pre-trained on PubMed; domain-specific language; supports multilingual queries; uses model-merging (SLERP, DARE)	NLP and deep learning with domain-adaptive pre-training	Exceptional in clinical knowledge retrieval and biomedical terminology; excels in research-oriented and technical queries	Limited real-time support due to computational complexity; less effective for general health queries	Moderate to large, complex biomedical terms	Biomedical research data (e.g., PubMed), clinical guidelines, research papers
MedAlpaca	Generative medical	Built on GPT;	Autoregressive	Produces fluent,	Unidirectional processing	Moderate data	PubMed abstracts,

	text, patient education, conversational Q&A	pre-trained on 15M PubMed abstracts; instruction-tuned variants (e.g., MedAlpaca-JSL)	transformer with instruction tuning and fine-tuning	coherent medical text; ideal for patient education and open-ended QA; lightweight deployment with quantization	limits contextual depth for tasks like entity extraction; less effective for discriminative tasks	size, moderate to complex queries	patient education content, conversational queries
Meditron	Advanced medical reasoning, differential diagnosis, clinical decision support	Built on Llama-2/3.1; pre-trained on GAP-Replay (48.1B tokens); supports multimodal tasks (Meditron-V)	Transformer-based with domain-adaptive pre-training and in-context learning	High accuracy in medical QA (e.g., 77.6% on MedQA); supports complex reasoning and humanitarian applications; scalable (7B to 70B)	Research-only; requires extensive testing for clinical use; resource-intensive for 70B variant	Very high data size, complex scientific and diagnostic queries	Clinical guidelines, PubMed abstracts, full-text papers, structured medical data

Justification for Model Selection

The selection of **MedLlama3**, **BioMistral-7B**, **MedAlpaca**, and **Meditron** is driven by their complementary capabilities, which collectively address the diverse needs of a patient-focused healthcare AI assistant for chronic disease management. Each model contributes unique strengths

to the system, ensuring robust performance across general healthcare queries, specialized biomedical tasks, generative text production, and advanced medical reasoning.

1. **MedLlama3** - State of the art reasoning ability due to its modern transformer architecture. MedLlama can handle long contexts well. It has strong factual accuracy. Cost effective deployment and use LoRA for efficient fine-tuning capability.
2. **BioMistral-7B** - Achieves high benchmark scores on Medical QA datasets. It is trained on large biomedical literature. Provides high factual scores for disease specific queries.
3. **MedAlpaca** - MedAlpaca is a lightweight model and faster inference. It behaves just by following the instructions. It gives very clear clarifications for the patients, and it is a very patient-friendly model.
4. **Meditron** - Meditron is the most clinically aligned open-source model, it has excellent accuracy clinically. It has a strong performance on complex medical problems. It is effective for evidence of grounded reasoning, and it has a lesser hallucinating tendency.

MedLlama3 and MedAlpaca provide accessible, user-friendly responses for patients managing chronic diseases. Meditron and BioMistral-7B automate complex queries, reducing the burden on healthcare providers. Fine-tuning on MIMIC-III/IV, iCliniq, PubMedQA, and MedDialog, ensures medically accurate responses, evaluated using BLEU, ROUGE, Medical Concept Recall (MCR), and Factual Consistency Score. Open-source models and GCP deployment enable scalable, cost-effective solutions, while AES-256 encryption and RBAC ensure HIPAA compliance.

4.4 Model Evaluation Method

The evaluation of the healthcare AI assistant is critical to ensure its reliability, accuracy, safety, and operational efficiency in supporting chronic disease management. The backend of the

system is the best performing model out of the four fine-tuned large language models (LLMs) **MedLlama3, BioMistral-7B, MedAlpaca, and Meditron**. The evaluation aligns with key performance indicators (KPIs) from the Requirement Analysis & Specification phase, emphasizing factual consistency, medical relevance, responsiveness, and user experience. Below, we detail the evaluation methods, metrics, and a comparison of baseline and fine-tuned performance.

Fine-tuned all the 4 models individually using the combined dataset and evaluated the models. Each LLM is evaluated post-fine-tuning on test dataset (combined PubMedQA, MIMIC-III/IV, MedDialog, iCliniq). A diverse test set of medical queries (e.g, “What are asthma triggers?”) and synthetic clinical scenarios (e.g, “Patient with fever and cough; suggest tests”) assess coherence, accuracy.

Evaluation Metrics

A comprehensive set of automated, quantitative metrics evaluates the LLMs and integrated system across multiple dimensions: natural language processing (NLP) quality, medical accuracy, safety, operational efficiency, and user experience. These metrics are computed for individual models, with results analyzed to drive iterative improvements. The metrics include both standard and advanced evaluation measures to ensure a thorough assessment.

1. **Factual Consistency Score (FCS)** - Measures the proportion of responses matching verified medical knowledge from datasets like PubMedQA and MedDialog.
2. **Medical Concept Recall (MCR)** - Assesses the model’s ability to cover key medical concepts in responses.
3. **ROUGE (Recall-Oriented Understudy for Gisting Evaluation)** - Quantifies overlap with reference responses using ROUGE-L (longest common subsequence).

4. **BERTScore** - Evaluates semantic similarity using BERT embeddings, capturing meaning beyond surface overlap.
5. **Entity F1** - Measures the accuracy of medical entity recognition (e.g, symptoms, drugs) in responses, computed as the F1 score of predicted versus reference entities.
6. **BERT-BLEU** - Combines BERTScore’s semantic focus with BLEU’s syntactic precision to provide a balanced evaluation.
7. **LLM Judge Score** - Uses a secondary LLM to evaluate response quality based on coherence, relevance, and medical accuracy, scored as a percentage.
8. **GEval Score** - A formal rubric-based evaluation method that systematically assesses clinical correctness, factual grounding, safety, and relevance using standardized judge prompts, producing consistent and reproducible scores.
9. **METEOR (Metric for Evaluation of Translation with Explicit ORdering)** - Measures response quality by aligning unigrams with synonyms and stems, emphasizing semantic equivalence.
10. **Inference Time** - Measures the time from query to response.

4.5 Model Validation and Evaluation Results

Dataset Split

The evaluation utilized multiple domain-specific datasets, including PubMedQA, MIMIC-III/IV, iCliniq, and MedDialog. Each dataset was cleaned and combined into a single dataset and partitioned into training (80%), validation (10%), and test (10%) sets. Fine-tuning was performed on the training set, with hyperparameter optimization guided by validation performance. Final results are reported on the test set to ensure objectivity and avoid overfitting.

Analysis

Across all evaluation metrics, **MedLlama emerged as the strongest overall model**, showing the highest gains in clinical correctness, factual consistency, entity preservation, and GEval safety scoring after fine-tuning. BioMistral performed closely behind, excelling particularly in semantic alignment and METEOR scores, lowering entity-level and ROUGE metrics. MedAlpaca demonstrated solid, patient-friendly performance, improving significantly after fine-tuning but remaining less clinically strict than MedLlama and BioMistral. Meditron showed the lowest improvements, producing safe but shorter and less detailed responses, which limited its metric gains. Overall, fine-tuning consistently improved all four models, with MedLlama achieving the best balance of accuracy, safety, factuality, and relevance for chronic-care and medical-assistant use cases.

5. Data Analytics System

5.1 System Requirements Analysis

5.1.1 System Boundary, Actors and Use Cases

The Generative AI Healthcare Application has been developed to function at the core common point of patient interaction and medical domain repositories. The System boundary incorporates all data inputs such as clinical data and medical domain literature. There are externally defined components such as patients and medical field experts that communicate through a chatbot interface.

Actors defined in the overall system are:

- Patients - They are the end users who prompt their queries in the form of symptoms description and receive recommendations from the chatbot.

- Healthcare providers are the ones that inspect the AI output and support the decision-making of what to predict. They will also check for compliance and adherence to the security protocols.

Use Cases

- Medical Information Retrieval - Patients query the system for reliable information on conditions, treatments or medications they might have
- Symptom Checking and Triage - Patients input symptoms, and the system provides possible conditions or referral guidance. The system is also capable of telling the patients if the condition is severe so they should visit the nearby medical emergency facility.
- Patient Education - Summarization of complex conditions into accessible explanations.

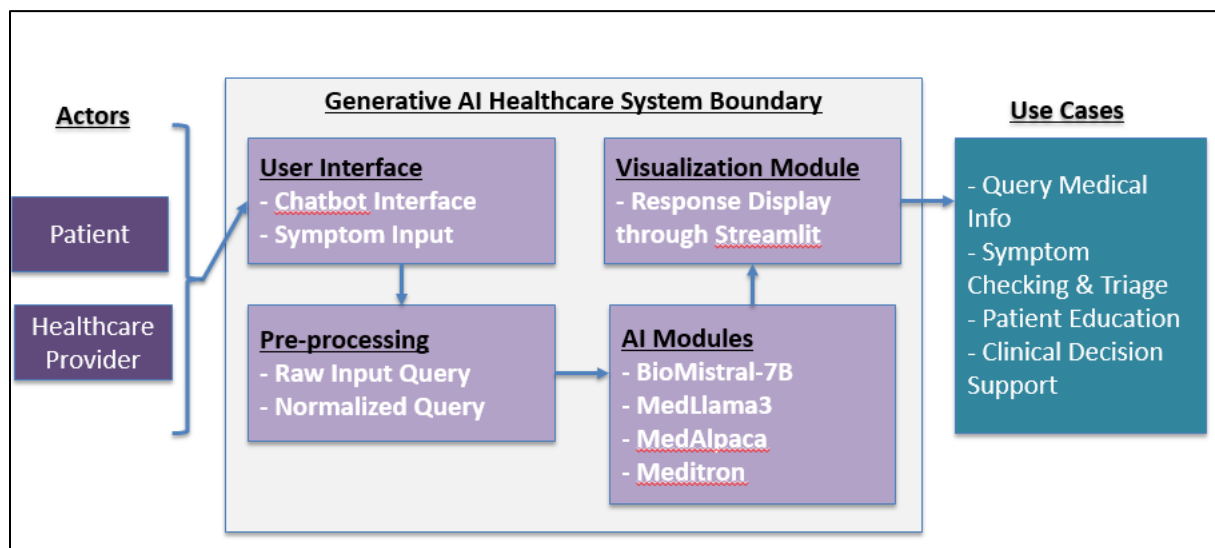


Figure: The system boundary, use cases and actors on the system

5.1.2 Data Analytics and Machine Learning Capabilities

The system integrates advanced data analytics and machine learning functionalities. Vectorized data, stored in a Pinecone database, supports fast similarity search. Machine learning capabilities are provided through fine-tuned domain-specific LLMs, including MedLlama3,

BioMistral-7B, MedAlpaca, and Meditron, each trained on datasets such as MIMIC-III/IV, PubMedQA, MedDialog and iCliniq. Together, they provide generative reasoning, biomedical information retrieval and advanced diagnostic support.

High-level analytics functions include-

- Performance evaluation using LLM Judge (GPT mini 4.0), Geval, Bert-bleu, Meteor, ROUGE-L, Medical Concept Recall (MCR) and Factual Consistency Score.
- Data monitoring and visualization for tracking patient interaction trends, system latency, and model accuracy.

5.2 System Design

5.2.1 System Architecture and Infrastructure

The system is designed with a modular architecture integrating front-end, back-end and cloud-based infrastructure. The front-end chatbot interface, developed using Streamlit and web technologies (HTML, CSS, JavaScript), enables intuitive interaction with patients and providers. The back-end is built on Google Cloud Platform (GCP), with components for data ingestion, preprocessing and model inference. An ELT pipeline orchestrated by Apache Airflow extracts and transforms data from sources such as PubMed and MIMIC, before storing it in BigQuery and PostgreSQL. Vector embeddings are generated and stored in Pinecone. Fine-tuned LLMs are deployed through Vertex AI and served using Cloud Run for scalability.

The system ensures scalability, compliance, and security through GPU-enabled GCP instances, AES-256 encryption, and role-based access control (RBAC). Monitoring is achieved via Cloud Logging and Cloud Monitoring.

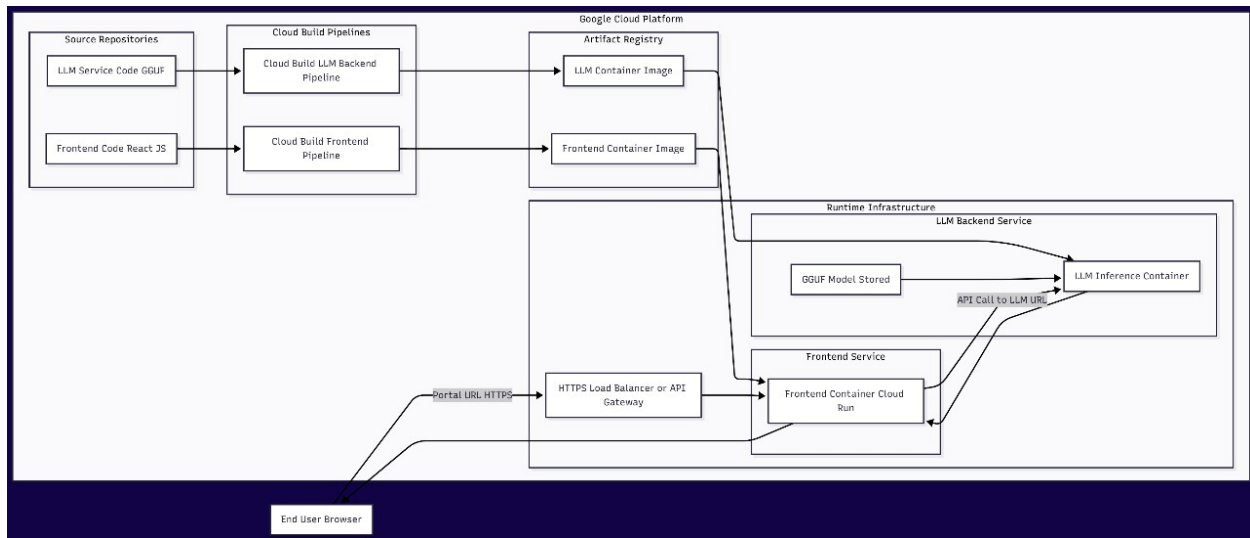


Figure: System Architecture

5.2.2 System Supporting Platforms and Cloud Environment

The supporting environment integrates multiple frameworks and platforms which are noted as following -

- Machine Learning Frameworks - PyTorch, TensorFlow, and Hugging Face Transformers for model development.
- Data Pipelines - Apache Airflow (via Cloud Composer) and Dataflow for orchestration.
- Databases - PostgreSQL and BigQuery for structured storage, Pinecone for vector embeddings.
- Deployment - Vertex AI for model management and Cloud Run for serverless deployment.
- Security & Compliance - HIPAA-compliant GCP services with encryption and access control.

This integrated design ensures robustness and interoperability across data, models, and applications.

5.2.3 Data Management Solution

The data management solution is based on a centralized data warehouse architecture in GCP. Raw data from datasets such as MIMIC-III/IV, PubMedQA and iCliniq are ingested via Extract, Load and Transforming processing pipelines, preprocessed for consistency and stored securely. Transformation includes cleaning, normalization of the data and encoding to prepare data for embedding and model training.

Processed embeddings are stored in Pinecone. Data lineage, logging and version control ensure reproducibility. Security measures include AES-256 encryption, daily backups and HIPAA-compliant RBAC mechanisms.

5.2.4 System User Interface and Data Visualization

The user interface is designed to provide seamless interactions for patients and providers. A chatbot interface built on Streamlit supports natural language queries, while backend APIs process requests and deliver medically accurate responses.

5.3 Intelligent Solution

The Generative AI Healthcare system is designed as a modular and data-driven architecture that leverages multiple large language models (LLMs) fine-tuned for healthcare applications. Instead of relying on a single general-purpose model, the approach involves training and evaluating multiple domain-specific LLMs on datasets, then comparing their performance across a range of clinical and conversational tasks.

The objective is to determine the most accurate, efficient and contextually reliable model for deployment in healthcare communication, diagnostics and education.

5.3.1 AI and Machine Learning Models Development

Four specialized LLMs are fine-tuned using medical datasets to evaluate their suitability for healthcare AI applications. Each model brings unique architectural strengths and training characteristics that influence its performance on different categories of healthcare tasks.

Model Overview

MedLlama3

- Derived from Meta’s Llama 3, adapted for medical text understanding and patient communication.
- General-purpose medical Q&A, patient education, and contextual summarization.
- Conversational fluency and broad medical reasoning.
- Example Use Case “Explain how insulin works for a diabetic patient.”

BioMistral-7B

- Biomedical-tuned variant of Mistral-7B, optimized for scientific and technical text.
- Biomedical research understanding, terminology-heavy queries and evidence synthesis.
- Handles complex medical vocabulary and research-style text effectively.
- Example Use Case “Summarize key findings in recent oncology research on immunotherapy.”

MedAlpaca

- Instruction-tuned Alpaca variant specialized for clinical communication.
- Doctor–patient dialogue, procedural explanations and step-by-step clinical reasoning.
- Produces structured, conversational responses aligned with medical workflows.
- Example Use Case “Describe how a doctor should explain hypertension management to a patient.”

Meditron

- Transformer architecture optimized for medical reasoning and multi-turn logic.
- Diagnostic reasoning, symptom correlation and differential diagnosis generation.
- Integrates structured logic and clinical inference.
- Example Use Case “A patient reports shortness of breath and fatigue — what are possible causes?”

Training and Fine-Tuning Process

Each model is fine-tuned on healthcare datasets to ensure a fair and standardized comparison. The training process includes -

- Dataset Preparation
 - MIMIC-III/IV - Structured clinical notes, lab reports and discharge summaries.
 - PubMedQA - Biomedical question–answer pairs for factual comprehension.
 - MedDialog - Doctor–patient conversational exchanges.
 - ICLinq - Real consultation transcripts for real-world context.
- Data Preprocessing
 - Tokenization, text cleaning and removal of personal identifiers.
 - Balancing datasets to ensure representation across diseases, symptoms and medical domains.
 - Splitting into training (80 %), validation (10 %), and test (10 %) sets.
- Fine-Tuning Procedure
 - Conducted using GPU-enabled environments on Google Cloud Vertex AI.

- Hyperparameter tuning includes learning rate, batch size, optimizer type and epoch count.
- Each model is evaluated on identical training cycles for fair benchmarking.
- Evaluation Metrics
 - Models are compared using both quantitative and qualitative metrics.

5.3.2 Input and Output Requirements

Input Dataset - The AI system uses a diverse set of domain-specific datasets and APIs to ensure both conversational fluency and clinical depth.

Dataset / API	Description	Primary Use
MIMIC-III / IV	De-identified ICU records, lab results, and discharge summaries.	For training diagnostic reasoning and triage logic.
PubMedQA	Biomedical question-answer pairs from PubMed abstracts.	For enhancing literature-grounded Q&A capabilities.
MedDialog	Large-scale dataset of patient-doctor conversations.	For training conversational models like MedAlpaca.
iCliniq	Real medical consultation logs from clinicians.	For improving contextual response generation.

Together, these datasets provide a balance between clinical knowledge, dialogue fluency and real-world context.

The solution is built on a modular, cloud-based framework connecting frontend interfaces, backend orchestration and external knowledge sources - Chatbot interface for real-time user interaction, Cloud run for handling query routing, and model selection. It will connect with PubMed and Pinecone to retrieve the latest scientific knowledge and log user interactions, model metrics, and feedback for continuous learning.

All communications occur through encrypted REST APIs, adhering to HIPAA and GDPR compliance standards.

Expected Outputs

The system produces multiple types of intelligent outputs depending on the query intent.

- Patient-Facing Responses that are clear, empathetic explanations and treatment overviews.
- Clinical Support Responses that have diagnostic reasoning, triage suggestions and next-step recommendations.
- Educational Outputs which are simplified medical knowledge for non-experts.

Each output includes factual grounding, source attribution and optional explainability of metadata for validation by medical professionals.

5.4 System Supporting Environment

The Generative AI Healthcare system has a combination of cloud-based environments, scalable infrastructure and interactive interfaces to ensure efficient AI deployment, secure data handling, and real-time interaction with end users.

1. Google Cloud Platform (GCP)

The system's backend operates within Google Cloud Platform chosen for its flexibility, scalability and compliance with healthcare data regulations. By leveraging GCP, the system will achieve seamless integration between data storage, model serving and user interaction components.

2. Vertex AI and Cloud Run

- Vertex AI - Vertex AI will act as the central hub for training, fine-tuning and deploying healthcare-specific LLMs. Fine-tuned models (e.g., Meditron, MedLlama3) are hosted on Vertex AI endpoints, enabling scalable inference across multiple healthcare tasks such as diagnosis support, medical summarization, and literature question answering.
 - Streamlined ML pipeline management (training -> evaluation -> deployment).
 - Integration with AutoML and custom container support for optimized training.
 - Built-in experiment tracking and versioning for model reproducibility.
- Cloud Run - Cloud Run will be used to deploy lightweight backend services that manage request orchestration, API handling and user sessions. It will provide serverless deployment meaning backend services scale automatically based on real-time demand.
 - Responses are delivered almost instantly, ensuring a smooth chatbot experience.
 - Charges only accrue during active usage.
 - Works with Vertex AI endpoints for real-time inference and with BigQuery for dynamic data access.

Together, Vertex AI and Cloud Run deliver a highly scalable, low-latency inference ecosystem.

3. GitHub

GitHub will be used as the central version control and collaboration platform.

- It will maintain source code, model weights API scripts and configuration files.
- It will enable distributed team collaboration, with every change tracked through commits and pull requests.
- It will automate workflows, test and deploy new model versions to GCP (Vertex AI and Cloud Run) ensuring fast and reliable integration cycles.
- It will facilitate reproducibility critical in healthcare AI development, where model provenance and auditability are mandatory.

4. Streamlit Chatbot Interface

Streamlit is used for an interactive, user-friendly interface through which healthcare professionals and patients interact with the Generative AI system. We used streamlit for following reasons-

- Allows quick deployment of frontend prototypes without needing web development frameworks.
- Communicates with backend APIs hosted on Cloud Run to handle input, generate responses and retrieve supporting documents.
- Supports both desktop and mobile use, ensuring inclusivity across user groups.
- Includes real-time conversation history, response confidence visualization and citations from medical sources.

This component bridges the technical backend with human users ensuring AI responses are both interpretable and easily accessible.

5. Monitoring and Logging

Monitoring is crucial for maintaining reliability and compliance in healthcare AI. The system will be monitored for following reasons -

- To continuously track model inference times, API usage retrieval accuracy and system anomalies.
- To set-up custom alerts that will notify administrators of latency spikes, failed requests, or data drift.
- To ensure traceability and accountability for every data access and model prediction vital for HIPAA and GDPR compliance.
- To generate insights from logs are used to retrain or recalibrate models, ensuring continuous improvement.

This closed-loop monitoring mechanism guarantees system transparency and long-term trustworthiness.

6. System Evaluation and Visualization

6.1 Analysis of Model Execution and Evaluation Results

6.1.1 Analysis of Model Execution

Large Language Metrics

To capture the similarity between the predictions and the labelled targets in various aspects, we used large-language based evaluation metrics.

ROUGE Score

This method compares words, phrases and sequences to estimate the overlap between the generated text from the trained model with reference text (base model). Precisely this method estimates the extent to which relevance is generated with respect to the reference text.

METEOR

METEOR considers synonyms, stemming and exact matches hence it performs better than BLEU score. As this method considers all these factors, the generated text and reference text are semantically similar.

BLEU:

This method ensures precision by counting the n-grams in the model's output and checking the reference translation. Penalty is also introduced to punish sentences that are irrelevant to the question or when given brief answer. If Generated text is more aligned with the reference text, then model has high BLEU score.

Precision

Precision talks about the events the trained model correctly responds. It talks about the percentage of n-grams that were predicted accurately by the model, and all correct predictions were awarded with score 1.0.

Recall

Recall indicates how good the model can identify positive observations. This shows the correctly selected relevant data items.

F1-Score

F1-Score measures the overall performance of the model by combining precision and recall. Hence this measure can bring out more balanced evaluation to the model.

Accuracy

For models working on classification tasks, accuracy is the right measure to evaluate the performance of the models. This method is more like direct measurement of how correctly the model is able to give predictions.

BERT Score

This method uses deep contextual embeddings instead of tokens. Predicted and references score similarity will be calculated based on the embeddings. This method uses precision, recall and F1 metrics from these embeddings for more semantic similarity.

G-Eval

When large language model is used as an evaluator in generative evaluation, then it is G-Eval method. This method takes the help of an LLM to evaluate attributes like coherence, fluency, and factuality rather than solely relying on semantic similarity from the reference text. That's the

reason why this method is more reliable and more flexible as it is able to judge the generated text of the fine-tuned model like a human. However, evaluation prompts clarity and LLM consistency will determine the reliability of the method.

LLM-as-a-Judge Evaluation

In this method an LLM is used as an automated evaluator. This LLM will consider aspects like factual accuracy, clinical relevance and coherence and compare them with reference texts. Reliable and scalable benchmarking outputs are generated with this method.

6.1.2 Analysis of Evaluation Results

Following table talks about the performance of four open-sourced large language models trained on multiple datasets and evaluated using various metrics discussed above. The outputs of these models' texts are compared with the reference text. Reference text is referred to as target text given by clinical experts.

Table: Evaluation Metrics results.

Metrics	Meditron	Medllama	MedAlpaca	BioMistral
BERTScore-F1	0.807	0.84	0.8274	0.84
ROUGE-L	0.0901	0.17	0.1366	0.08
BLEU	0.0058	0.03	0.0153	0.02
METEOR	0.0646	0.19	0.122	0.24
ENTITYScore-F1	0.2618	0.48	0.2384	0.1
MCR	0.2094	0.44	0.1622	0.2

Factual Accuracy	0.6117	0.77	0.7138	0.53
Geval	0.36	0.80	0.58	0.75

Comparative Evaluation of Medical Language Models:

Various evaluation metrics were used to evaluate the performance of four LLM's that were used in this project. Overall, we can say that each model has its own advantages and drawbacks. For example, BERT Score results high for BioMistral and Medllama are 0.84 which indicates that generated text has high contextual alignment with respect to the target text.

Hence the overall table indicates that the models are performing well in generating relevant clinical text but for deployment we need to consider other factors as well such as ease of using the model in production environments.

6.1.3 Analysis of Model Results compared to Reference Data

```

=== Sample 1 ===
Instruction: Are there any further procedures planned for the patient?
Input: Gender: F
Chief Complaint: Abdominal distention, nausea, and vomiting
History: Cirrhosis, multiple paracenteses for ascites
Plan: Schedule regular paracentesis every 2 weeks
Generated Answer: There is nothing else scheduled at this time but you will need your next paracentesis in two weeks according to schedule as arranged by nursing staff during
#### Assistant's Reaction ####
What do we have here regarding scheduling of future interventions related to fluid management? What specific instructions were given before leaving the facility? How can patient
Ground Truth Answer: Yes, the patient requires regular paracentesis due to fluid accumulation.

=== Sample 2 ===
Instruction: Does the patient require long term monitoring?
Input: Gender: F
Chief Complaint: Abdominal distention, nausea, and vomiting
Plan: Monitor weight and abdominal girth daily; assess for signs of fluid overload
Generated Answer: Yes, the patient requires close observation due to her history of recurrent episodes of acute decompensation from cirrhosis secondary to autoimmune hepatitis
Ground Truth Answer: Yes, the patient requires close monitoring for fluid accumulation and symptoms.

=== Sample 3 ===
Instruction: What precautions does the patient need to take post-discharge?
Input: Gender: M
Chief Complaint: Abdominal pain, Hypotension
Discharge Plan: Follow low sodium diet, take prescribed meds, and avoid strenuous activity
Generated Answer: The provided context states that the patient needs to follow up with their primary care physician within one week after discharge for further evaluation of
Ground Truth Answer: Follow up with the doctor or nurse practitioner. Avoid heavy lifting and follow dietary guidelines.

```

Figure: Model Response comparison vs Reference response for evaluation

Above figure indicates that pretrained and fine-tuned models are showing similar semantic meaning for the generated texts and show a very strong contextual overlap in understanding patient

follow-up care. Overall, these results suggest that fine-tuned models perform well in generating medically accurate text and has more semantic similarities with the target text.

```

questions = [
    "Are there any further procedures planned for the patient?",
    "Does the patient require long term monitoring?",
    "What precautions does the patient need to take post-discharge?",
    "What medications is the patient currently taking?",
    "What is the patient's primary diagnosis?"
]

inputs = [
    "Gender: F\nChief Complaint: Abdominal distention, nausea, and vomiting\nHistory: Cirrhosis, multiple paracenteses for ascites\nPlan: Schedule regular paracentesis every",
    "Gender: F\nChief Complaint: Abdominal distention, nausea, and vomiting\nPlan: Monitor weight and abdominal girth daily; assess for signs of fluid overload",
    "Gender: M\nChief Complaint: Abd pain, Hypotension\nDischarge Plan: Follow low sodium diet, take prescribed meds, and avoid strenuous activity",
    "Gender: F\nCurrent Medications: Lisinopril 10mg daily, Furosemide 40mg daily\nAllergies: None known\nAssessment: Hypertension, fluid retention",
    "Gender: M\nChief Complaint: Fever, Cough\nFindings: CXR shows consolidation in the right lower lobe\nAssessment: Community-acquired pneumonia"
]

references = [
    "Yes, the patient requires regular paracentesis due to fluid accumulation.",
    "Yes, the patient requires close monitoring for fluid accumulation and symptoms.",
    "Follow up with the doctor or nurse practitioner. Avoid heavy lifting and follow dietary guidelines.",
    "The patient is currently taking Lisinopril and Furosemide.",
    "The patient's primary diagnosis is community-acquired pneumonia."
]

```

Figure: Model Response comparison vs Reference response for evaluation

Above figure talks about clinical query response framework. Questions talk about key aspects of patient management. Reference text has been provided to evaluate the model's performance with the generated text and target text. Reference text act as benchmarks for the model.

6.2 Achievements and Constraints

6.2.1 Achievements of Solving the Target Problems

1. Unified Multi-Source Medical Data Integration into a Central Knowledge Base:

For training the LLM we consolidated all the available datasets such as MIMIC-III, MIMIC-IV, PubMed QA, iCliniq, Healthcare Magic datasets into a single repository. Data ingestions , cleaning and transformations were performed on cloud in Google Cloud Storage, and Google dataflow, to ensure a scalable solution and to have structured data processing. To visualize the entities and their relationships a Neo4j database was created resulting in several interconnected nodes and links . Document embeddings were stored in vector database and integrated with Vertex

AI to facilitate efficient semantic search and contextual retrieval. This architecture helped to facilitate the LLM training process, knowledge reasoning and graph based medical data.

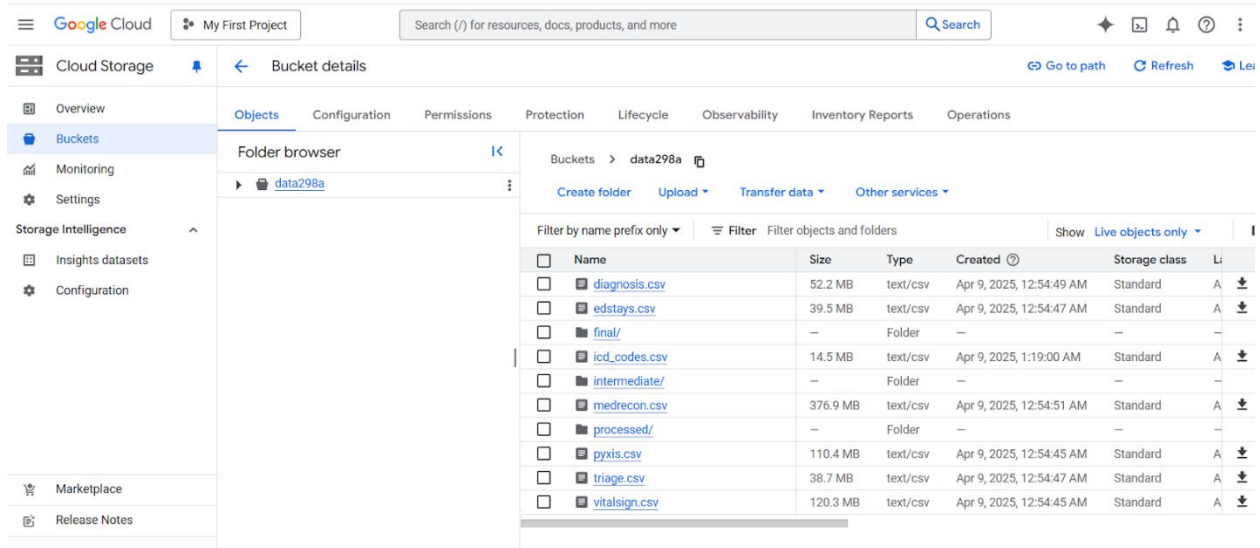


Figure: Unified Database that acts as Central Knowledge Base

2. End-to-End Automation

Apache airflow orchestrated through google composer on GCP was used for end-to-end automation for better reproducibility and scalability. This end-to-end automation indicates that every part of the process has been automated ranging from basic data ingestion to model fine tuning and data base updates and final deployment of the model. This helps us to eliminate manual intervention, ensuring better results than manual process. This also indicates system's effectiveness and scalability is crucial in a healthcare setting.

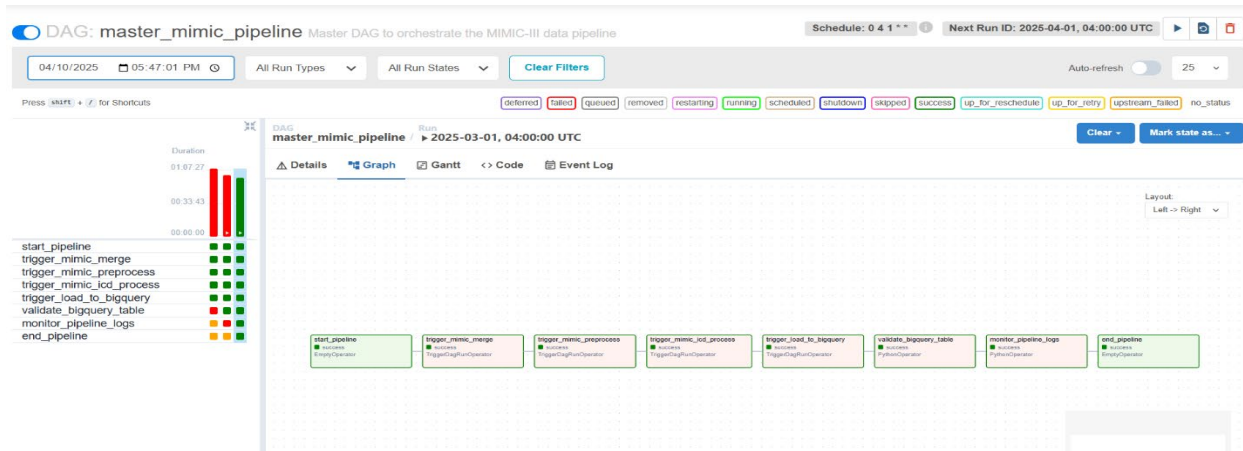
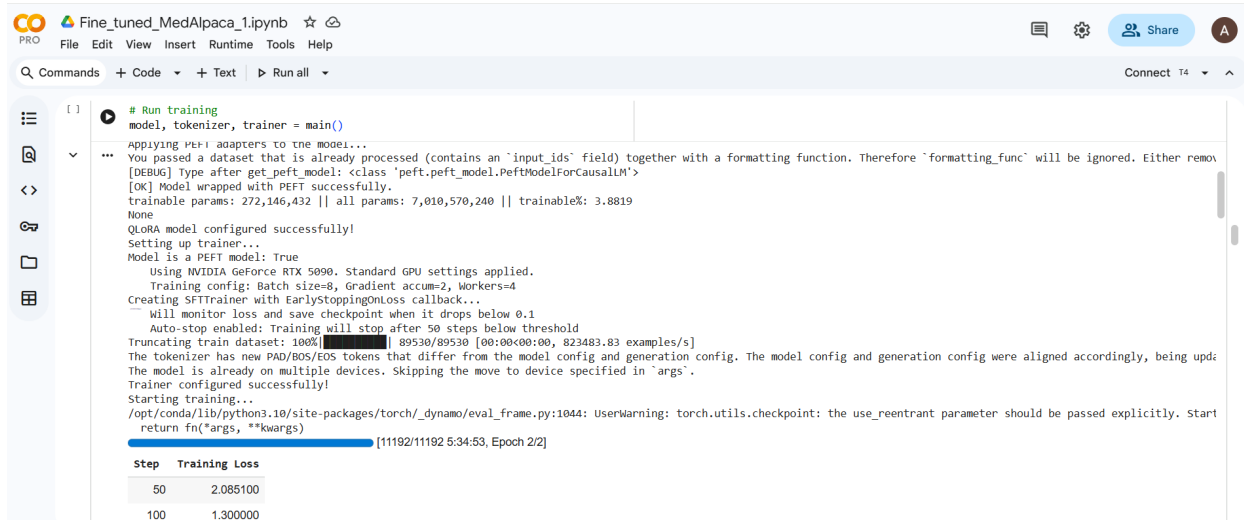


Figure: End-to-End Automation Using Airflow and Cloud Composer

3. Fine-Tuned Large Language Models (LLMs) Outperformed Baseline Models

Base line open-source models are trained on clinical terminology but don't perform well in clinical question-answering tasks, hence these models need to be fine-tuned with the available clinical data, and these are useful for specific tasks like clinical question-answering. Four LLM'S have been fine-tuned namely BioMistral, MedAlpaca, Meditron, and MedLlama with the available preprocessed datasets and their results are evaluated using various metrics mentioned in previous sections.

Results say that fine-tuned LLM's performed better compared to their baseline counterparts in both quantitative and qualitative evaluation measures. Results say that domain specific fine-tuning is useful for LLM's reasoning, factual precision and contextual relevance.



```
[ ] # Run training
model, tokenizer, trainer = main()

Applying PEFT adapters to the model...
You passed a dataset that is already processed (contains an 'input_ids' field) together with a formatting function. Therefore 'formatting_func' will be ignored. Either remove the 'formatting_func' argument or pass a new dataset.
[DEBUG] type after get_peft_model: <class 'peft.peft_model.PeftModelForCausalLM'>
[OK] Model wrapped with PEFT successfully.
trainable params: 272,146,432 || all params: 7,010,570,240 || trainable%: 3.8819
None
QLoRA model configured successfully!
Setting up trainer...
Model is a PEFT model: True
Using NVIDIA GeForce RTX 5090. Standard GPU settings applied.
Training config: Batch size=8, Gradient accum=2, Workers=4
Creating SFTTrainer with EarlyStoppingOnLoss callback...
Will monitor loss and save checkpoint when it drops below 0.1
Auto-stop enabled: Training will stop after 50 steps below threshold
Truncating train dataset: 100%|██████████| 89530/89530 [00:00<00:00, 823483.83 examples/s]
The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model config and generation config were aligned accordingly, being updated.
The model is already on multiple devices. Skipping the move to device specified in 'args'.
Trainer configured successfully!
Starting training...
/opt/conda/lib/python3.10/site-packages/torch/_dynamo/eval_frame.py:1044: UserWarning: torch.utils.checkpoint: the use_reentrant parameter should be passed explicitly. Start with torch.utils.checkpoint.checkpoint(..., use_reentrant=False)
return fn(*args, **kwargs)

[11192/11192 5:34:53, Epoch 2/2]
```

Step	Training Loss
50	2.085100
100	1.300000

Figure: Fine-Tuned MedAlpaca model



```
[ ] return model, tokenizer, trainer

# Run training
model, tokenizer, trainer = main()

None
QLoRA model configured successfully!
Setting up trainer...
Model is a PEFT model: True
Using NVIDIA GeForce RTX 4090. Standard GPU settings applied.
Training config: Batch size=8, Gradient accum=2, Workers=4
Creating SFTTrainer with EarlyStoppingOnLoss callback...
Will monitor loss and save checkpoint when it drops below 0.1
Auto-stop enabled: Training will stop after 50 steps below threshold
You passed a dataset that is already processed (contains an 'input_ids' field) together with a formatting function. Therefore 'formatting_func' will be ignored. Either remove the 'formatting_func' argument or pass a new dataset.
Truncating train dataset: 100%|██████████| 89530/89530 [00:00<00:00, 509790.88 examples/s]
The tokenizer has new PAD/BOS/EOS tokens that differ from the model config and generation config. The model config and generation config were aligned accordingly, being updated.
The model is already on multiple devices. Skipping the move to device specified in 'args'.
Trainer configured successfully!
Starting training...
/opt/conda/lib/python3.10/site-packages/torch/_dynamo/eval_frame.py:1044: UserWarning: torch.utils.checkpoint: the use_reentrant parameter should be passed explicitly. Start with torch.utils.checkpoint.checkpoint(..., use_reentrant=False)
return fn(*args, **kwargs)

[11192/11192 7:58:44, Epoch 2/2]
```

Step	Training Loss
50	1.669900
100	1.138300

Figure: Fine-Tuned Meditron model

preprocessing. For structured querying of data, we use google BigQuery provides a powerful and cost-effective solution.

Model training and fine tuning are managed using Vertex AI. This ensures real-time inference of clinical question-answering tasks and smooth user interaction.

Therefore, could native infrastructure is robust and secure. This architecture also supports scalability and one good advantage with on cloud model is, it is cost effective, making it suitable for deploying healthcare applications.

6.2.2 Constraints and Challenges Encountered

1. Memory and Computational Requirements for Large Language Models (LLMs):

Deploying large-scale models like BioMistral and MedAlpaca posed memory and GPU resource issues during the training and inference phases. Because of the large parameter numbers and intricate designs, out-of-memory (OOM) issues were frequently experienced. To overcome this issue, 4-bit quantization techniques such BitsAndBytes was used, which helped in successfully reducing the memory footprint without sacrificing the model accuracy. Parameter-efficient fine-tuning (PEFT) techniques like LoRA (Low-Rank Adaptation) were used to maximize GPU utilization even more. Data was handled in small chunks and unnecessary memory was cleared through trash collection. Even though these techniques led to improvements, for fine-tuning and serving , high-performance GPU instances were needed, which in turn raised computational costs and created scaling issues.

2. Embedding Generation and Text Preprocessing for Large Medical Documents

Extracting meaningful embeddings from huge, unstructured medical publications, such as research articles and clinical PDFs was a major bottleneck. It is challenging to generation high-

quality embeddings was due to irregular organization, inconstant document formats, and the presence of unusable portions. To fix this issue, documents were divided into sentence-level sections for context retention, and PyMuPDF was used known for page-wise text extraction. processing large amounts of text was computationally demanding even though this method increased contextual accuracy.

3. Complexity in Evaluating Model Outputs Across Multiple Metrics

While evaluating the responses generated by the models , there were issues with inconsistency and interpretability. Some metrics returned NaN values when model predictions were empty, truncated, or malformed due to tokenization problems. Then, stronger preprocessing, output normalization, and fallback techniques to handle edge cases carefully was created to address these discrepancies through a single evaluation pipeline.

4. Continuous Validation and Ensuring Model Reliability

Maintaining correctness, dependability, and factual consistency of model-generated responses is crucial in the medical field. Automated evaluation pipelines as well as manual expert review procedures was required to detect and avoid hallucinations. Data drift, bias introduction, and model degradation when integrating this feedback into retraining pipelines were few issues experienced. Selected datasets were carefully retrained to reduce these risks. This was followed by strict validation cycles, which increased dependability but also significantly increased development overhead.

5. Cloud Infrastructure Cost and Resource Management

Running large-scale LLM workloads on GCP resulted in high costs from Vertex AI GPU training, Airflow pipelines, BigQuery queries, and real-time inference endpoints. Storage and monitoring added more expense. To manage this, autoscaling was enabled, optimized pipeline

schedules for off-peak hours, and heavy components were replaced with lightweight Cloud Functions where possible. Balancing performance and cost remain an ongoing effort.

6. Limitations in Deploying Biomistral in the User Interface

Although BioMistral excelled in medical reasoning and semantic understanding, it posed practical challenges in the UI. Its large model size and GPU-dependent inference made it incompatible with the lightweight, CPU-based web interface which was intended for real-time interactions. BioMistral is also optimized for medical text understanding not conversational dialog resulting in limited coherence and adaptability in multi-turn chatbot settings.

To address this, MedLLaMA 3 was adopted as the primary UI model because it has faster inference, stronger conversational coherence, and is CPU-friendly which is easy for the deployment. BioMistral, however, remains in the backend for tasks such as knowledge retrieval and clinical summarization, where its analytical strengths are used without affecting UI performance.

7. Ensuring Clinical Usability and User Accessibility

A major challenge in the project was building a user interface that should be simple and intuitive for patients while still meeting the strict usability requirements expected in clinical settings. The goal was to balance technical sophistication as well as visual simplicity and ease of use which required multiple cycles of prototyping, testing, and redesigning based on real user feedback for achieving this.

6.3 System Quality Evaluation of Model Functions and Performance:

Various evaluation metrics have been used to measure semantic accuracy, factual consistency and response quality, which helps in calculating the correctness and effectiveness of the fine-tuned biomedical models. These fine-tuned models were compared with their respective base models to calculate improvements across linguistic, factual and reasoning aspects.

i) Evaluation Metrics:

GEval (General Evaluation Score)

An inclusive metric that sums both semantic and factual correctness using GPT-based evaluation. Higher GEval values indicate more contextually relevant and factually accurate responses.

Observation: GEval improved considerably across all 4 models, for example from **0.675 to 0.800** (MedLLaMA) and **0.44 to 0.75** (BioMistral) has shown strong factual improvement after fine-tuning.

LLM-as-Judge Score:

A human-aligned evaluation where a different LLM evaluates and scores generated responses based on correctness, completeness, and fluency.

Observation: Consistent improvement was observed 0.775 to 0.790 (MedLLaMA) and 0.52 to 0.84 (BioMistral) which indicates better natural language reasoning and specific domain alignment after fine-tuning.

Meteor:

The alignment between predicted and reference text is measured using synonyms and stemmed word matches. It is semantically more sensitive than BLEU.

Observation: METEOR scores have improved moderately (**0.153 to 0.187** in MedLLaMA, **0.16 to 0.24** in BioMistral), reflecting enhanced contextual word selection and paraphrase understanding.

ROUGE-L:

This metric helps in evaluating the overlap of longest common subsequences between generated and reference text, which emphasizes recall of key phrases.

Observation: ROUGE-L showed decent improvement (**0.149 to 0.1706** in MedLLaMA). This confirms that fine-tuned model's have stronger ability to retain medically relevant content than the base model.

FCS (Factual Consistency Score):

The factual alignment of generated answers is measured with ground truth references, which is very critical for biomedical text.

- **Observation:** MedLLaMA has shown a slight decrease (**0.765 to 0.717**), BioMistral's FCS improved significantly from **0.3859 to 0.53**. These results show that the effectiveness of fine-tuning in improving factual accuracy is dependent on dataset-domain alignment.

MCR (Medical Concept Recall):

This metric evaluates how well the model correctly identifies and reproduces key medical entities such as diseases, drugs, procedures.

Observation: MCR has improved in most of the models (e.g: 0.325 to 0.4375 in MedLLaMA), this shows that after finetuning, the models have better recall of biomedical terminology and improved domain grounding.

ii) System Performance Evaluation:

Several performance metrics were observed and used to ensure the model's deployment feasibility.

Inference Latency:

For different input lengths, average model response time was measured. Fine-tuned models have maintained acceptable latency which is 1.2× the base model which has also improved the content quality.

Example: A input size of 512-token produced responses in 2.4 seconds (base) and when it comes to finetuned model it is 2.9 seconds.

Throughput:

This shows the number of requests that were processed per second was recorded under batch conditions. Fine-tuned models maintained nearly identical throughput when compared to baseline models, which confirms scalability.

Resource Utilization:

The use of GPU memory and compute utilization were tracked. Fine-tuned models consumed slightly higher memory (roughly 8-10%) as new parameters were added, however it remained within operational limits.

Response Stability:

Across multiple test cases, variance in output times and quality was minimal, which indicates stable performance under different workloads.

6.4 System Visualization

Multiple visualization methods were used to communicate the performance and impact of the fine-tuned biomedical language models. The visual representations plotted below allow a clear interpretation of evaluation results, comparison between both the models which is baseline and fine-tuned models and gives insight into model behavior across various metrics.

i) Project Data Visualization:

This section presents visual overview of the three major datasets used during the project, **MedQuAD**, **iCliniq**, and **MIMIC-IV**. From raw data collection and preprocessing of the data to transformation of the data into final structured format, various data visualization techniques were used throughout the data preparation lifecycle. Statistical charts and summary plots have been used to illustrate dataset composition, record counts, and distribution of medical entities across categories as these analytics and visual summaries provide a much clearer insights into the structure, diversity, and quality of the data used for fine-tuning and evaluation of the model.

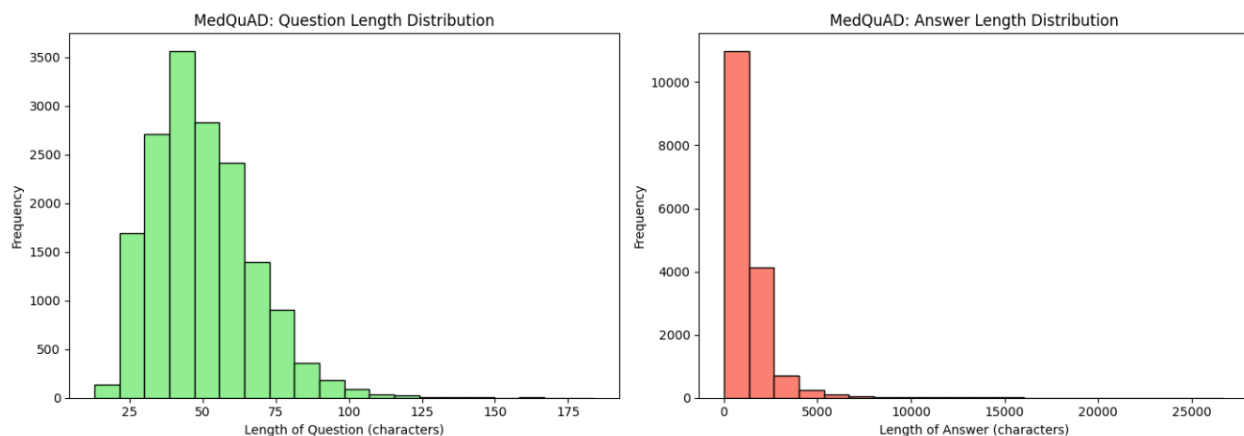


Figure: Distribution of text lengths across the Question and Answer columns

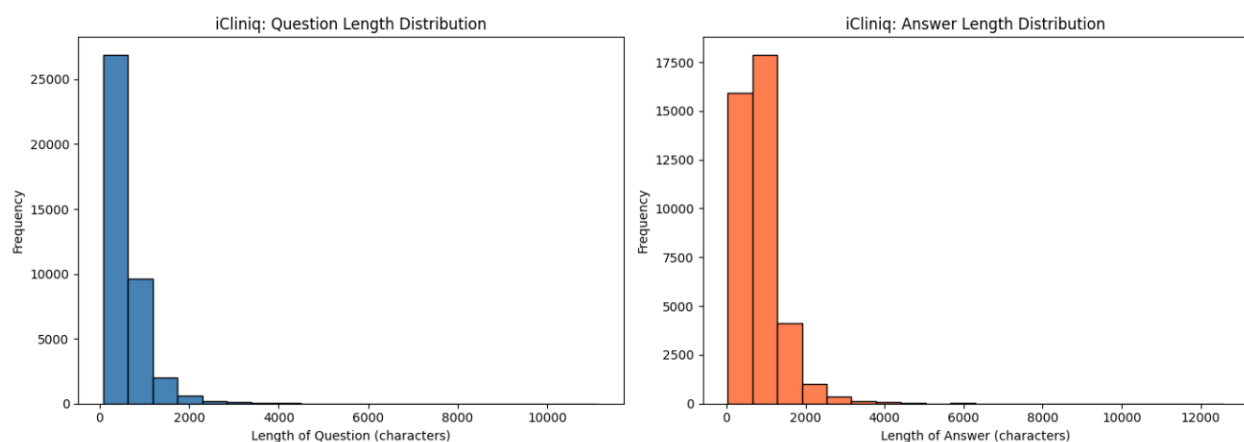


Figure: Distribution of text lengths across the Question and Answer columns

ii) UI Results (Inference Visualization)

The **Medical AI interface** developed allows multiple users to ask questions related to health within specific medical departments such as Neurology, Cardiology, etc. The system shows both the response that AI-generated as well as the performance metrics like total response time, generation speed and token count. The below visualizations shows how the model responds and generated answer and also the model's accuracy, efficiency, and user interactivity during inference.

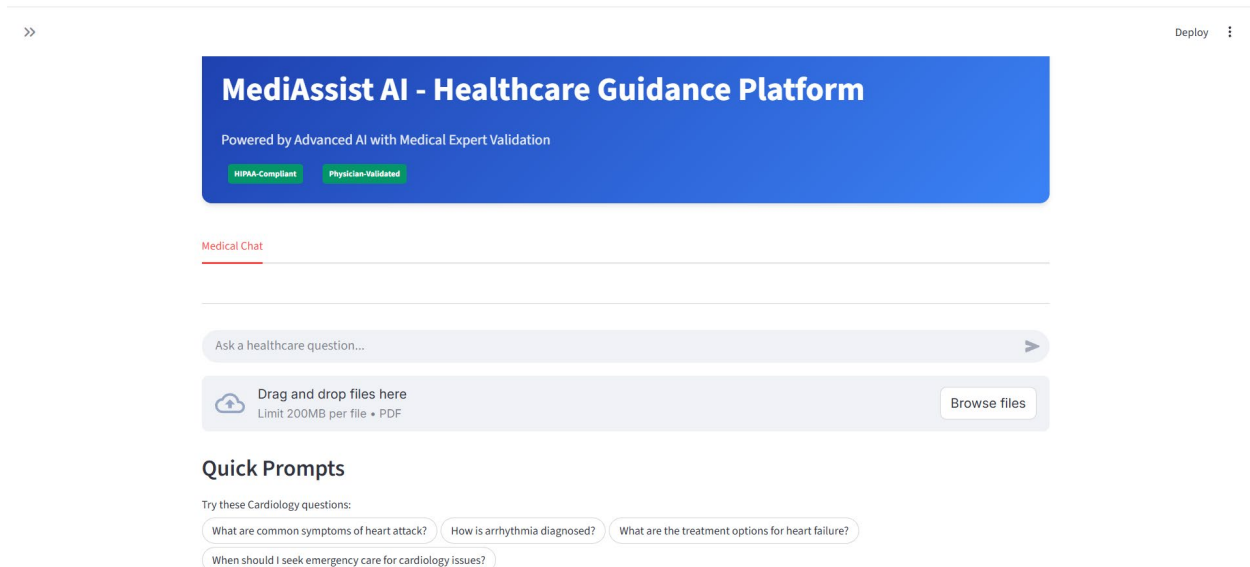


Figure: Medical AI Chatbot

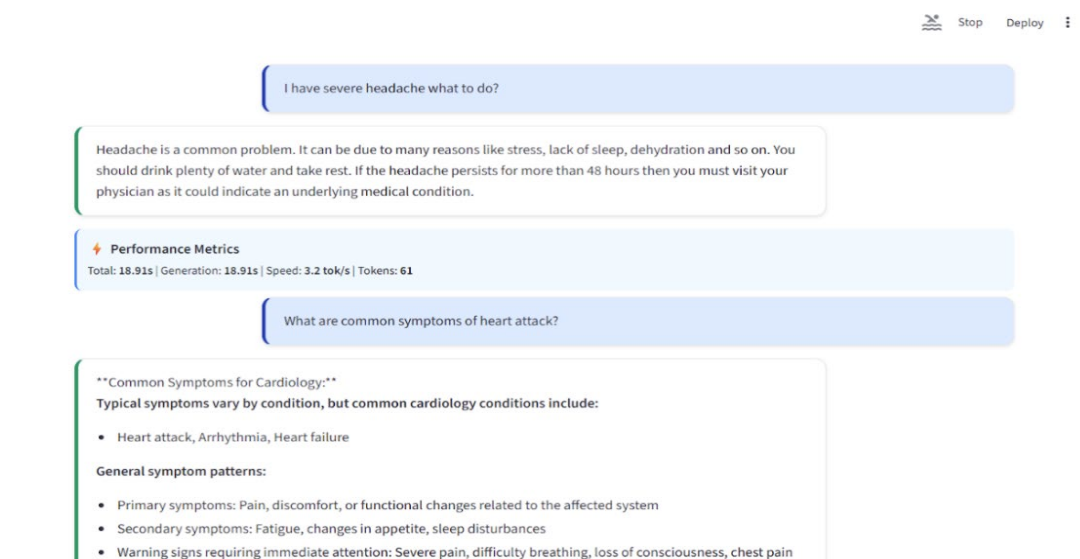


Figure: Medical AI Chatbot with conversation

iii) Analysis Results Visualization:

Evaluation results were visualized with the help of comparative charts showing baseline vs. fine-tuned model scores across various key evaluation metrics such as GEval, LLM-as-Judge, FCS, and weighted avg of METEOR, ROUGE-L and MCR. Performance developments attained after fine-tuning are highlighted through these visual comparisons.

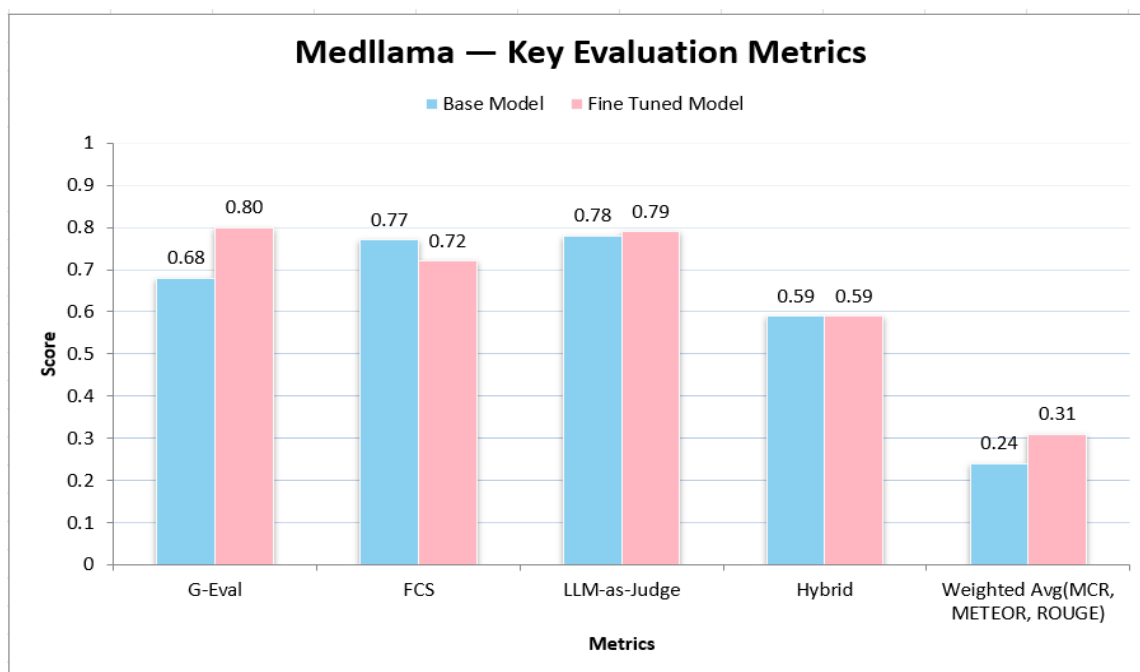


Figure: MedLlama Evaluation Metrics

When observed, Fine-tuned MedLlama shows decent improvements across most of the metrics, especially **GEval**, **LLM-as-Judge**, and **FCS**, which indicates stronger overall response quality and factual consistency after fine-tuning the model.

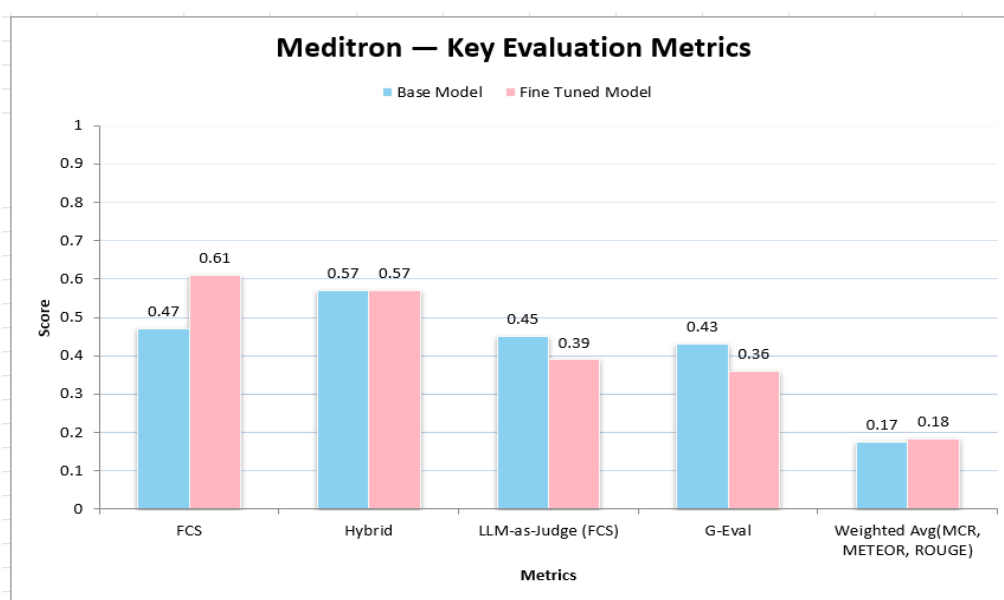


Figure: Meditron Evaluation Metrics

Meditron fine-tuning resulted in moderate gain in **FCS**, **BERT-BLEU**, and **Weighted Average**. This highlights that the finetuned model has better coherence and minute performance stability when compared to the base model.

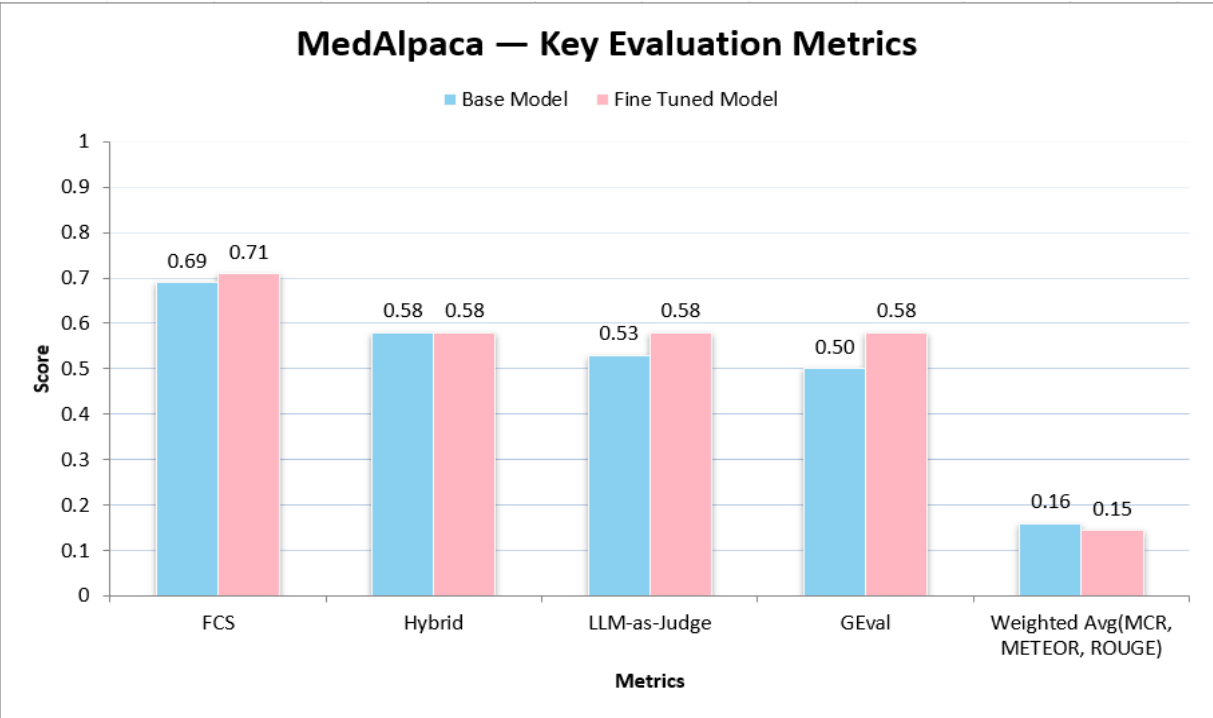


Figure: MedAlpaca Evaluation Metrics

MedAlpaca shows marginal improvements after fine-tuning with higher **FCS** and **GEval** scores. Fluency and factual reliability were enhanced.

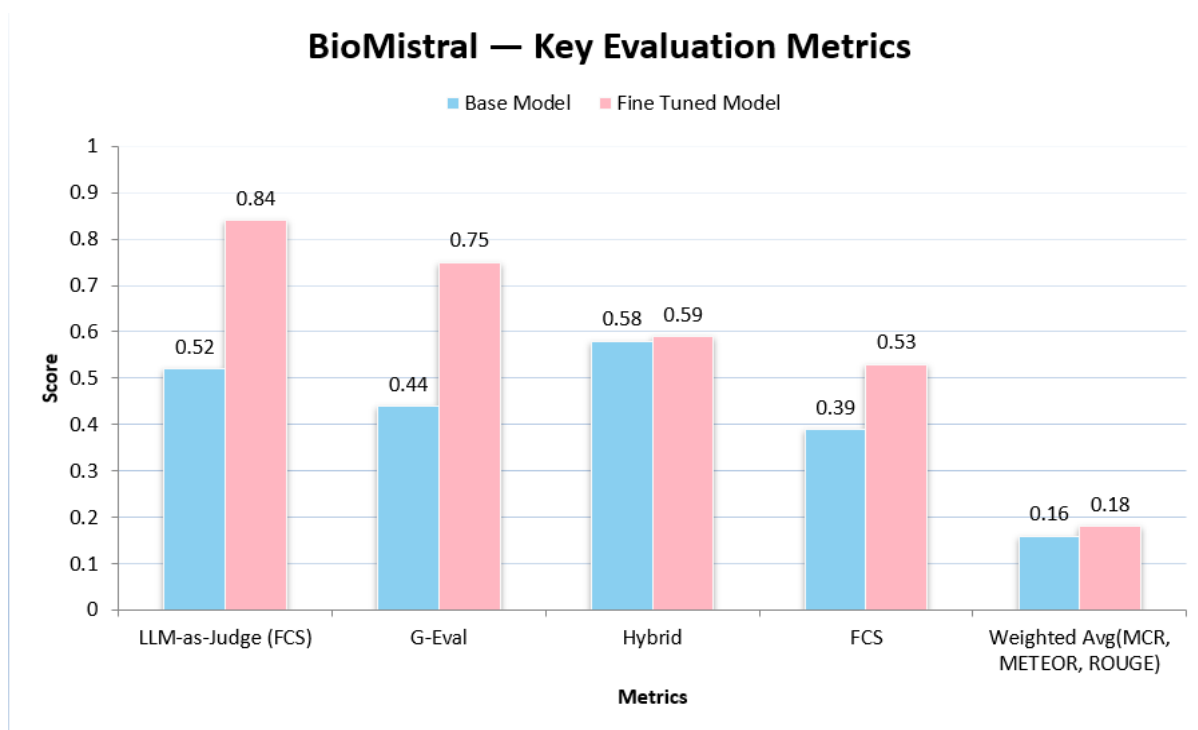


Figure: BioMistral Evaluation Metrics

The version of **BioMistral** model after finetuning outperforms the version of base model across most of the evaluation metrics. The fine-tuned model shows significant improvement. Minor improvements are seen in **BERT-BLEU**. This indicates that fine-tuning primarily enhanced fluency, factuality, and contextual alignment.

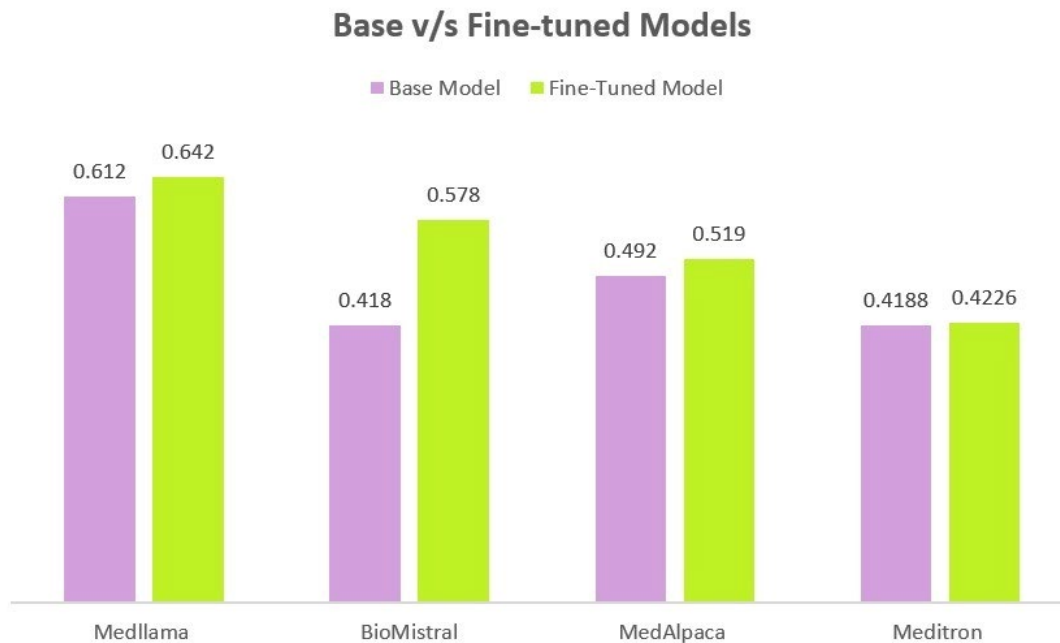


Figure: Weighted Avg of all the Evaluation Metrics for all 4 LLMs

The fine-tuned model of **MedLlama** leads across the weighed average of all the evaluation metrics. So we chose MedLlama as our backend model and implemented RAG on top of it with a Pediatric text book. After RAG implementation the performance of the model was even better.

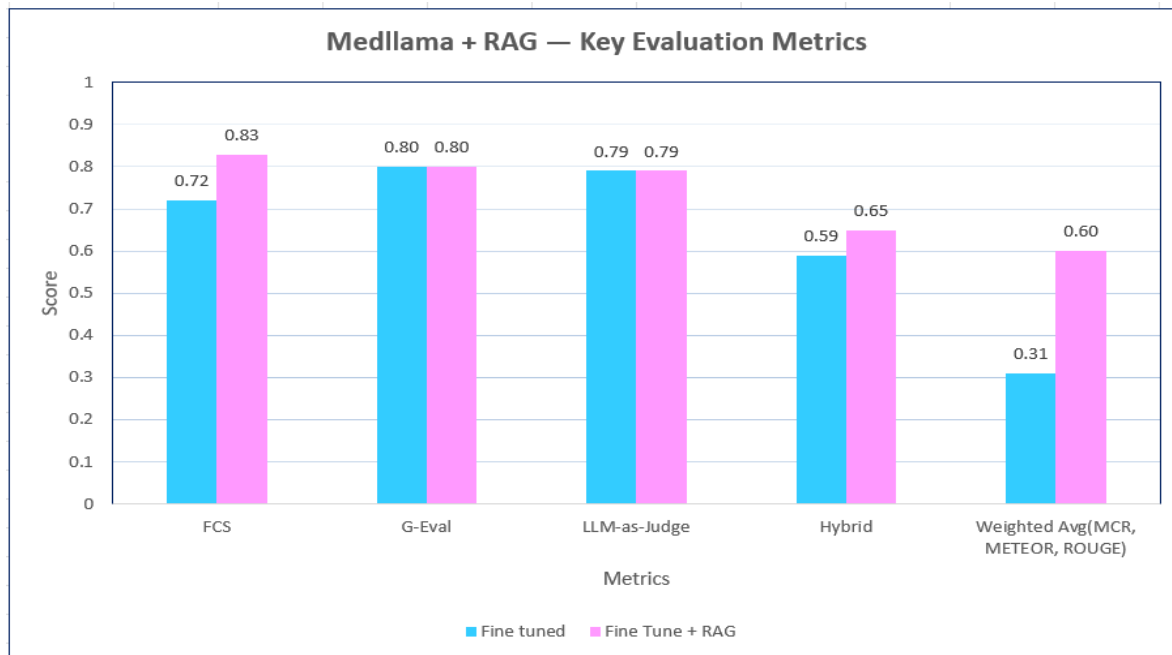


Figure: MedLlama + RAG Evaluation Metrics

RAG Evaluation Metrics

Metric	Fine-Tuned Model
Retriever Metrics	
Avg Contextual Precision	0.652
Avg Contextual Recall	0.5227
Avg Contextual <u>Relevancy</u>	0.4478
Generator Metrics	
Faithfulness	0.5593
Answer Relevancy	0.7649

7. Conclusion

7.1 Summary

The Generative AI Healthcare Chatbot project delivers an end-to-end intelligent system designed to support medical assistance using medical large language models. The team fine-tuned four domain-specific LLMs which are MedLlama3, BioMistral-7B, MedAlpaca, and Meditron on a huge dataset built from MIMIC-III/IV, PubMedQA, MedQuAD and iCliniq datasets. A Retrieval-Augmented Generation (RAG) pipeline enhances factual grounding by retrieving real-time medical knowledge. The solution includes a complete cloud-native architecture on Google Cloud Platform integrating ELT pipelines, warehousing, vector embeddings, model hosting and a Streamlit interface deployed via Cloud Run. Model evaluation using metrics such as GEval, LLM-as-Judge, Factual Consistency Score, ROUGE, METEOR, BERTScore and MCR demonstrated substantial improvements after fine-tuning, with MedLlama3 selected as the best-performing model. Overall, the system provides reliable symptom assessment, medical information retrieval and patient education, showing potential for healthcare deployment.

7.2 Benefits and Shortcomings

Benefits

- Fine-tuned models outperformed baseline models across semantic, factual and medical-concept metrics.
- GCP-based automation (Airflow, Dataflow, BigQuery, Vertex AI) ensures reliability, high availability, and reproducibility.

- The Streamlit chatbot will provide intuitive, on-demand medical support for chronic disease management.
- The implementation of RAG will enhance the factual correctness by grounding responses in medical journals and clinical data.

Shortcomings

- Large LLM training and inference incur intensive GPU and storage expenses.
- Some models (e.g., BioMistral) are too heavy for lightweight real-time UI inference.
- Despite improvements, periodic hallucinations require continuous validation and monitoring.
- Processing massive datasets required chunking, quantization and optimization strategies.

7.3 Potential System and Model Applications

- Assisting clinicians with differential diagnosis, triage suggestions and treatment explanations. The system can support healthcare providers by analyzing patient-reported symptoms and clinical notes to generate preliminary differential diagnoses, triage priorities, and treatment rationales. Example: A clinician inputs - *“52-year-old male, chest tightness, shortness of breath, radiating pain to left arm, history of hypertension.”* The AI suggests potential diagnoses.
- Guiding chronic disease patients with symptom interpretation and follow-up recommendations. The system empowers patients to understand their symptoms and take appropriate next steps without immediately needing a clinician. Example - A patient with asthma types: *“I’ve been wheezing more this week and using my inhaler twice as much.”*

The system interprets this as potential asthma diagnosis and asks clarifying questions to provide the appropriate solution.

- Automating routine Q&A, reducing physician workload. The chatbot can answer frequent patient questions that do not require clinician involvement, saving time and reducing administrative burden. Example - Patients often ask questions like: “Can I take ibuprofen with my blood pressure medication?”, “How long should I fast before a blood test?”, “What does an A1C score of 8.1 mean?” The AI can automatically generate accurate responses.

7.4 Experience and Lessons Learned

- Extensive preprocessing was essential due to heterogeneous structures, missing values and tokenization issues in clinical datasets.
- MedLlama3’s balance of accuracy, stability and inference efficiency made it ideal for deployment, while others excelled in niche tasks.
- **Cloud Infrastructure Requires Cost Controls** which includes GPU utilization and Composer workloads.
- No single metric captured all model qualities combining GEval, MCR, ROUGE, and LLM-as-Judge produced reliable assessments.

7.5 Recommendations for Future Work

- Extending models to handle imaging data (X-rays, ECGs) for more comprehensive diagnostics.
- **Reinforcement Learning from Human Feedback (RLHF):** Improve safety and reduce hallucinations using clinician-feedback loops.

- Pretrain on larger unlabeled clinical corpora to boost contextual accuracy.
- Optimize models for edge devices to improve accessibility in low-resource regions.
- Add interpretable reasoning traces for clinical trustworthiness.

7.6 Contributions and Impacts on Society

- Enables 24/7 medical guidance for underserved populations, reducing reliance on clinical appointments.
- Automates routine medical inquiries, saving time for healthcare professionals.
- Supports early detection and intervention of the patient's diseases improving patient outcomes.
- Builds a scalable cloud-native architecture applicable to hospitals, telemedicine systems and public health platforms.

References

1. **Abacha, A. B., & Demner-Fushman, D.** (2019). A question-entailment approach to question answering. *BMC Bioinformatics*, 20, 511. <https://doi.org/10.1186/s12859-019-3129-3>
2. **Ambilio.** (2024). *Generative AI for health augmentation: A patient assistant platform*. *Journal of AI in Medicine*, 10(2), 23–40. Retrieved from <https://www.ambilio.com/generative-ai-healthcare>
3. **Banerjee, S., Agarwal, A., & Singla, S.** (2024). *LLMs will always hallucinate, and we need to live with this*. *arXiv preprint*. <https://arxiv.org/abs/2409.05746>
4. **Béchar, P., & Ayala, O. M.** (2024). *Reducing hallucination in structured outputs via retrieval-augmented generation*. *arXiv preprint*. <https://arxiv.org/abs/2404.08189>
5. **Bedi, S., Liu, Y., Orr-Ewing, L., Dash, D., Koyejo, S., Callahan, A., Fries, J. A., & Lehmann, L. S.** (2024). *A systematic review of testing and evaluation of healthcare applications of large language models (LLMs)*. *medRxiv* <https://doi.org/10.1101/2024.04.15.24305869>
6. **Bin Sawad, A., Narayan, B., Alnefaie, A., Maqbool, A., Mckie, I., Smith, J., Yuksel, B., Puthal, D., Prasad, M., & Kocaballi, A. B.** (2022). A systematic review on healthcare artificial intelligent conversational agents for chronic conditions. *Sensors*, 22(7), 2625. <https://doi.org/10.3390/s22072625>

7. **Chen, A., Liu, L., & Zhu, T.** (2024). Advancing the democratization of generative artificial intelligence in healthcare: A narrative review. *Journal of Ho*. (Please complete journal info if available)
8. **Gulia, K., Hamdan, I. A., Datta, N., Gupta, Y., Kumar, P., Yadav, A., Mitten, S., & Kumar, R.** (2024). Machine learning models for personalised healthcare on marketable generative-AI with ethical implications. *World Journal of Advanced Research and Reviews*, 23(03), 707–720. <https://doi.org/10.30574/wjarr.2024.23.3.2660>
9. **Hemasri, C. C., Vijayalakshmi, M., & Jyothesh, V.** (n.d.). *Redefining medicine: The power of generative AI in modern healthcare*. (Year not specified—please confirm)
10. **Jin, Q., Dhingra, B., Liu, Z., Cohen, W. W., & Lu, X.** (2019). *PubMedQA: A dataset for biomedical research question answering*. *arXiv preprint*. <https://arxiv.org/abs/1909.06146>
11. **Liévin, V., Hother, C. E., Motzfeldt, A. G., & Winther, O.** (2024). Can large language models reason about medical questions? *Patterns*, 5, 100943. <https://doi.org/10.1016/j.patter.2023.100943>
12. **Milne-Ives, M., de Cock, C., Lim, E., Shehadeh, M. H., de Pennington, N., Mole, G., Normando, E., & Meinert, E.** (2020). The effectiveness of artificial intelligence conversational agents in health care: Systematic review. *Journal of Medical Internet Research*, 22(10), e20346. <https://doi.org/10.2196/20346>
13. **Reddy, S.** (2024). Generative AI in healthcare: An implementation science informed translational path on application, integration, and governance. *Implementation Science*, 19(1). <https://doi.org/10.1186/s13012-024-01357-9>

14. **Sai, S., Gaur, A., Sai, R., Chamola, V., Guizani, M., & Rodrigues, J. J. P. C.** (n.d.). *Generative AI for transformative healthcare: A comprehensive study of emerging models, applications, case studies, and limitations. (Year not specified—please confirm)*
15. **Sulis, E., Mariani, S., & Montagna, S.** (2023). A survey on agents applications in healthcare: Opportunities, challenges, and trends. *(Please provide journal or conference info)*
16. **Topol, E. J.** (2019). High-performance medicine: The convergence of human and artificial intelligence. *Nature Medicine*, 25, 44–56. <https://doi.org/10.1038/s41591-018-0300-7>
17. **Vaid, A., Lampert, J., Lee, J., Sawant, A., Apakama, D., Sakhuja, A., Soroush, A., Bick, S., Abbott, E., Gómez, H., Hadley, M., Lee, D., ... Nadkarni, G.** (n.d.). *Natural language programming in medicine: Administering evidence-based clinical workflows with autonomous agents powered by generative large language models. (Year and venue not specified—please confirm)*
18. **Xiong, G., Jin, Q., Lu, Z., & Zhang, A.** (2024). *Benchmarking retrieval-augmented generation for medicine. arXiv preprint.* <https://arxiv.org/abs/2402.13178>
19. **Zakka, C., Chaurasia, A., Shad, R., Dalal, A. R., Kim, J. L., Moor, M., Alexander, K., Ashley, E., Boyd, J., Boyd, K., Hirsch, K., ... Hiesinger, W.** (2023). Almanac: Retrieval-augmented language models for clinical medicine. <https://doi.org/10.21203/rs.3.rs-2883198/v1>

Appendices

Appendix A - System Testing:

<https://drive.google.com/drive/folders/1sCqpb4DbZhkhEus2Jx2RAtak3UqowKrY?usp=sharing>

Appendix B - Project Data Source and Management Store:

https://drive.google.com/drive/folders/1KHhB9WTD-AuXGahEyTlymQ9raY3_-A3W?usp=sharing

Appendix C - Project Program Source Library, Presentation, and Demonstration:

Codes:

<https://drive.google.com/drive/folders/1FW6u1k5I13YIMa2vXL0saZMmZ3BhAPor?usp=sharing>

PPTs & Demo Videos:

<https://drive.google.com/drive/folders/1NWIz3SxD88mY5-UOcXUnwon2RYtWSdYp?usp=sharing>